

01.12.2025

Балансировка деревьев.

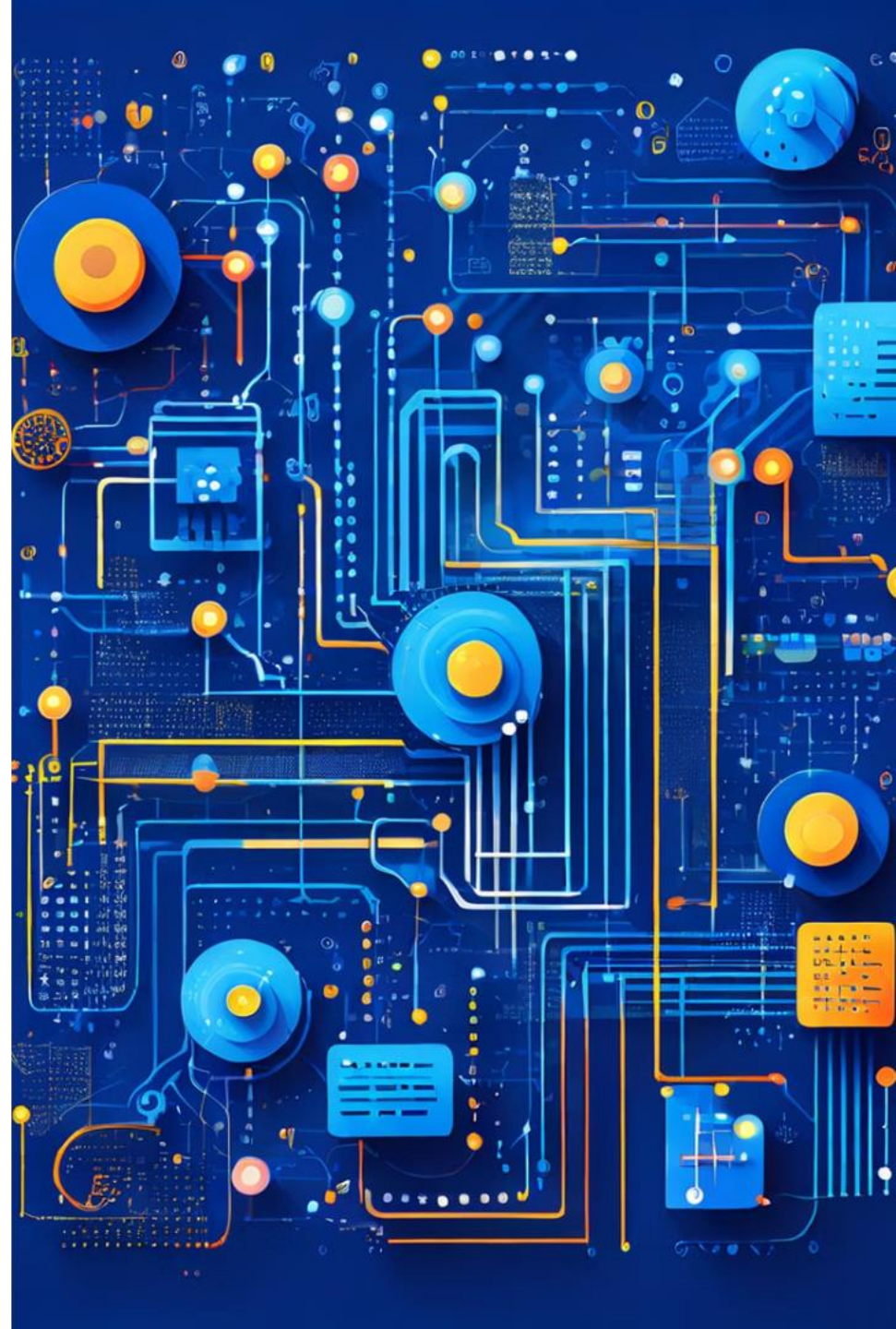
Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

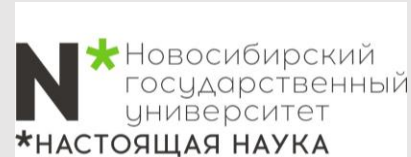


Давайте познакомимся

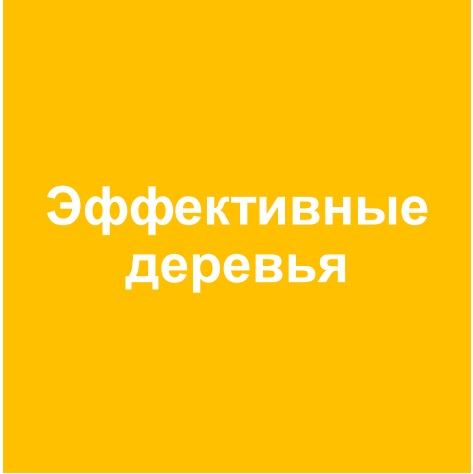


Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.



План лекции



Эффективные
деревья

70 минут

Дерева AVL

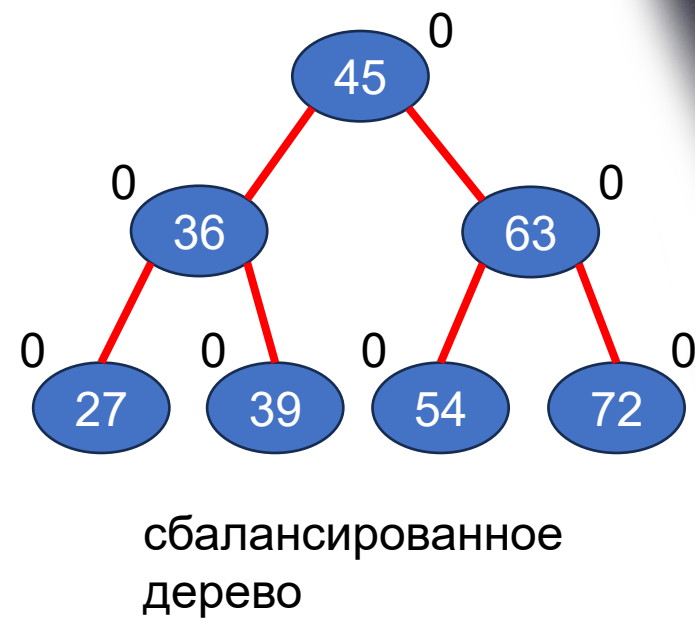
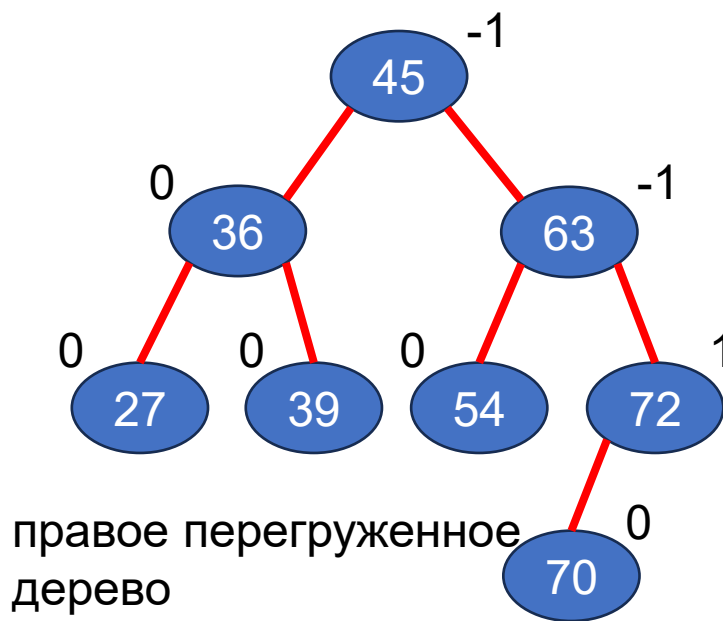
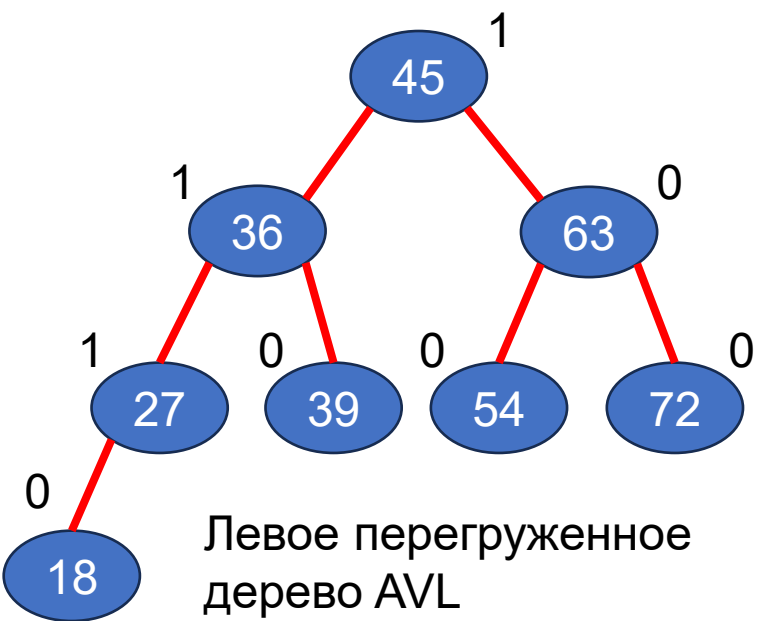
Дерево AVL — это самобалансирующееся бинарное дерево поиска, изобретенное Г. М. Адельсоном-Вельски и Э. М. Ландисом в 1962 году. Дерево названо AVL в честь своих изобретателей. В дереве AVL высоты двух поддеревьев узла могут отличаться не более чем на единицу. Благодаря этому свойству дерево AVL также известно как дерево со сбалансированной высотой. Главное преимущество использования дерева AVL заключается в том, что для выполнения операций поиска, вставки и удаления требуется время $O(\log n)$ как в среднем, так и в худшем случае, поскольку высота дерева ограничена $O(\log n)$. Структура дерева AVL такая же, как и у бинарного дерева поиска, но с небольшим отличием. В своей структуре оно хранит дополнительную переменную, называемую *BalanceFactor*. Таким образом, с каждым узлом связан фактор сбалансированности. Коэффициент балансировки узла вычисляется путем вычитания высоты его правого поддерева из высоты его левого поддерева. Двоичное дерево поиска, в котором каждый узел имеет коэффициент балансировки -1 , 0 или 1 , называется сбалансированным по высоте. Узел с любым другим коэффициентом балансировки считается несбалансированным и требует повторной балансировки дерева.

Коэффициент балансировки = Высота (левого поддерева) – Высота (правого поддерева)

- Если коэффициент балансировки узла равен 1 , то это означает, что левое поддерево дерева находится на один уровень выше, чем правое поддерево. Поэтому такое дерево называется деревом с перегрузкой влево.
- Если коэффициент балансировки узла равен 0 , то это означает, что высота левого поддерева (самый длинный путь в левом поддереве) равна высоте правого поддерева.
- Если фактор баланса узла равен -1 , то это означает, что левое поддерево дерева находится на один уровень ниже, чем правое поддерево. Такое дерево поэтому называется деревом с перегрузкой справа.

Дерева AVL

Деревья, представленные на рисунках, являются типичными кандидатами на деревья AVL, поскольку коэффициент балансировки каждого узла равен 1, 0 или -1 . Однако вставки и удаления из дерева AVL могут нарушить коэффициент балансировки узлов, и, таким образом, может потребоваться повторная балансировка дерева. Дерево перебалансируется путем выполнения поворота в критическом узле. Существует четыре типа поворотов: поворот LL, поворот RR, поворот LR и поворот RL. Тип поворота, который необходимо выполнить, будет зависеть от конкретной ситуации.



Поиск узла в дереве AVL

Поиск в дереве AVL выполняется точно так же, как и в бинарном дереве поиска. Из-за балансировки высоты дерева операция поиска занимает время $O(\log n)$. Поскольку операция не изменяет структуру дерева, никаких специальных положений не требуется.



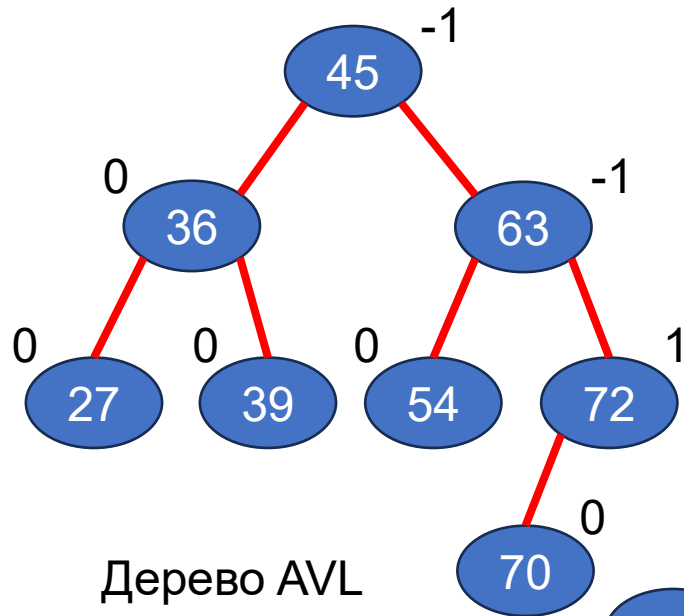
Вставка нового узла в дерево AVL

Вставка в дерево AVL также выполняется так же, как и в бинарное дерево поиска. В дереве AVL новый узел всегда вставляется как конечный узел. Но за шагом вставки обычно следует дополнительный шаг поворота. Поворот выполняется для восстановления баланса дерева. Однако, если вставка нового узла не нарушает фактор баланса, то есть если фактор баланса каждого узла по-прежнему равен -1 , 0 или 1 , то повороты не требуются. Во время вставки новый узел вставляется как конечный узел, поэтому он всегда будет иметь фактор баланса, равный нулю. Единственными узлами, коэффициенты балансировки которых изменятся, являются те, которые лежат на пути между корнем дерева и вновь вставленным узлом. Возможные изменения, которые могут произойти в любом узле на пути, следующие:

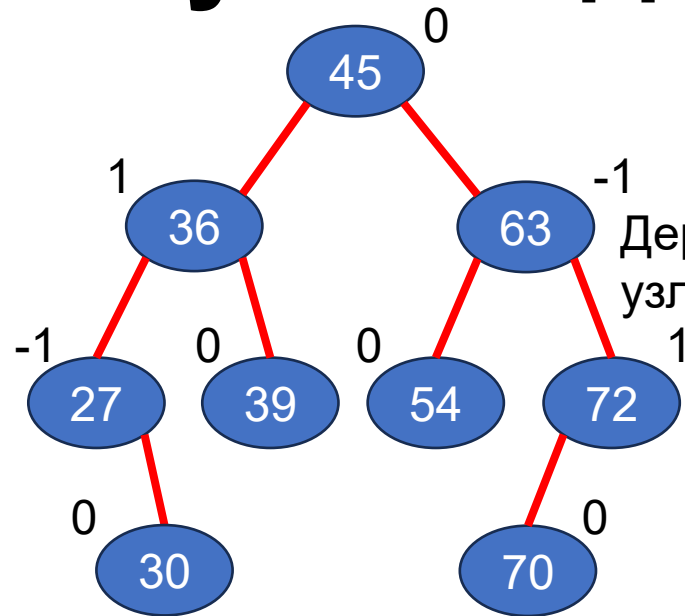
- Изначально узел был либо лево-, либо право-тяжелым, а после вставки он становится сбалансированным.
- Изначально узел был сбалансированным, а после вставки он становится либо лево-, либо право-тяжелым.
- Изначально узел был тяжелым (либо левым, либо правым), а новый узел был вставлен в тяжелое поддереву, тем самым создавая несбалансированное поддереву. Такой узел называется **критическим узлом**.



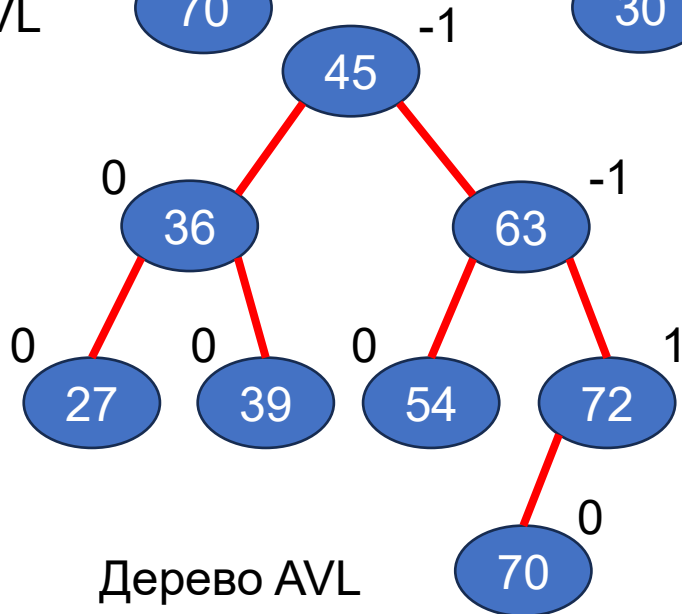
Вставка нового узла в дерево AVL



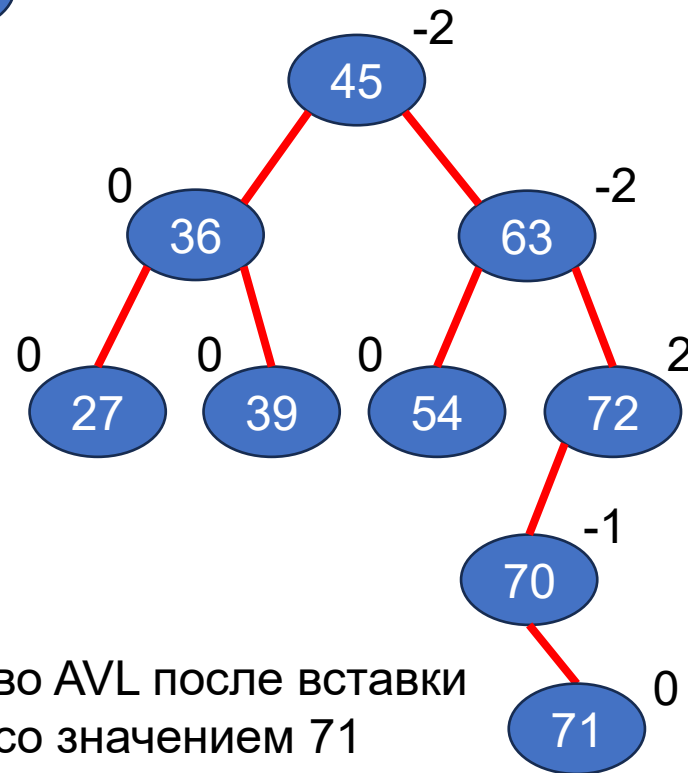
Дерево AVL



Дерево AVL после вставки узла со значением 30



Дерево AVL



Дерево AVL после вставки узла со значением 71

Вставка нового узла в дерево AVL

Итак, возникает необходимость выполнить поворот. Для выполнения поворота наша первая задача — найти критический узел. Критический узел — это ближайший узел-предок на пути от вставленного узла к корню, коэффициент балансировки которого не равен ни -1 , ни 0 , ни 1 .

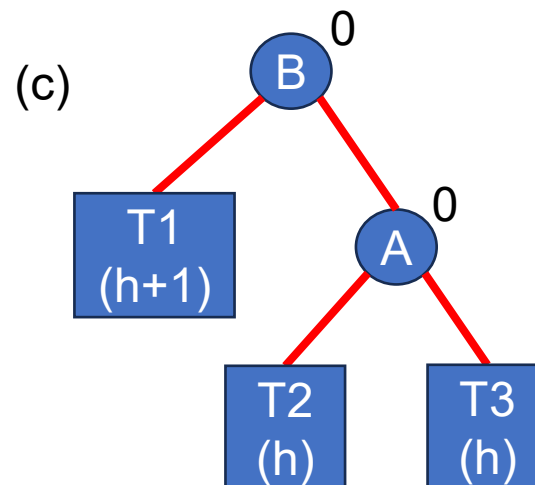
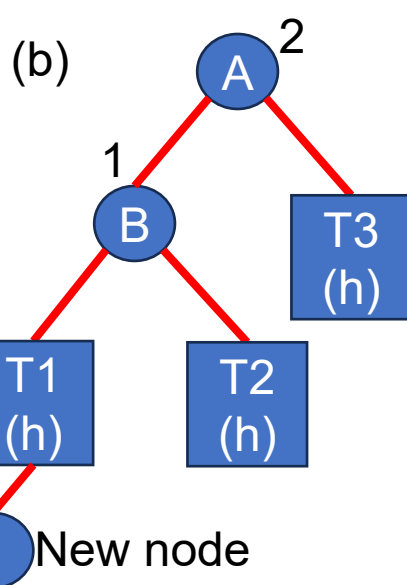
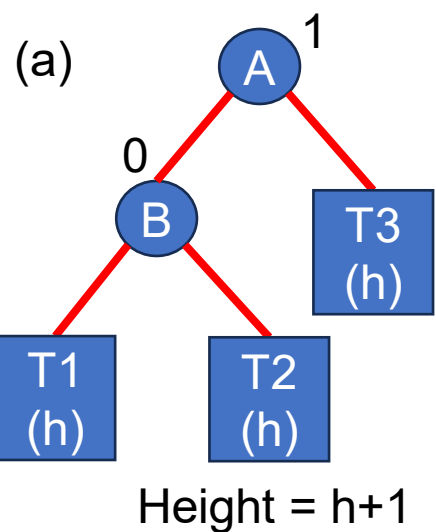
Вторая задача при перебалансировке дерева — определить, какой тип поворота необходимо выполнить. Существует четыре типа поворотов перебалансировки, и применение этих поворотов зависит от положения вставленного узла по отношению к критическому узлу. Четыре категории поворотов:

- **поворот LL** Новый узел вставляется в левое поддерево левого поддерева критического узла.
- **поворот RR** Новый узел вставляется в правое поддерево правого поддерева критического узла.
- **поворот LR** Новый узел вставляется в правое поддерево левого поддерева критического узла.
- **вращение RL** Новый узел вставляется в левое поддерево правого поддерева критического узла.



Вращение LL

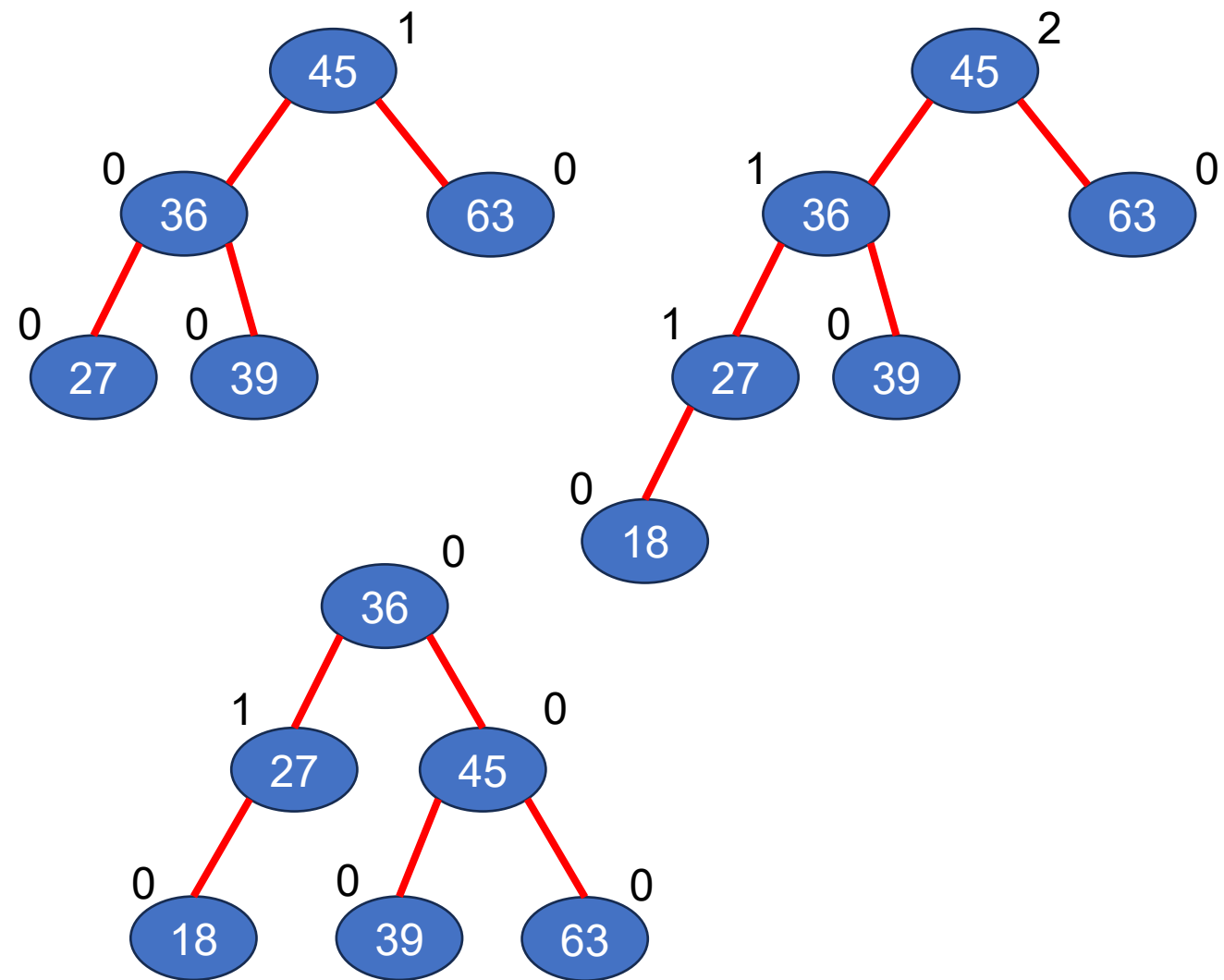
Давайте подробно изучим каждое из этих вращений. Сначала мы увидим, где и как применяется вращение LL. Рассмотрим дерево, приведенное на рисунке, которое показывает дерево AVL. Дерево (a) является деревом AVL. В дереве (b) новый узел вставляется в левое поддерево левого поддерева критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент баланса которого не равен -1 , 0 или 1), поэтому мы применяем вращение LL, как показано на дереве (c).



Вращение LL в дереве AVL

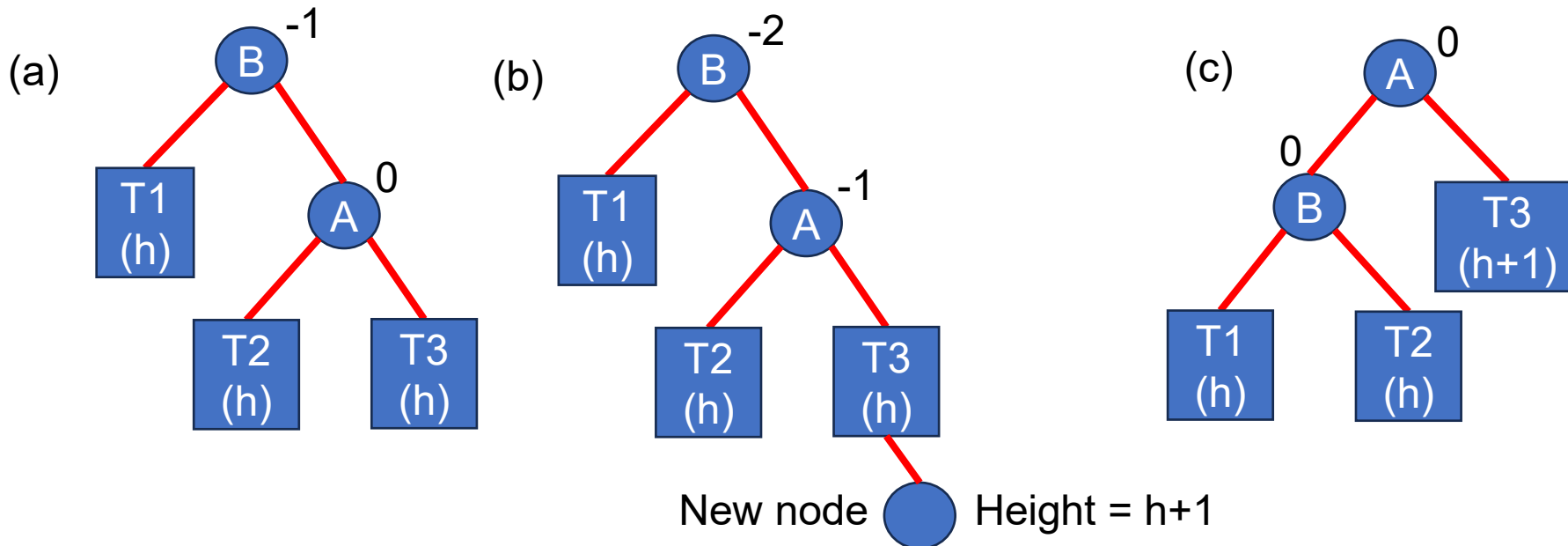
Во время вращения узел B становится корнем, а T1 и A являются его левым и правым потомками. T2 и T3 становятся левым и правым поддеревьями A.

Вращение LL - Пример

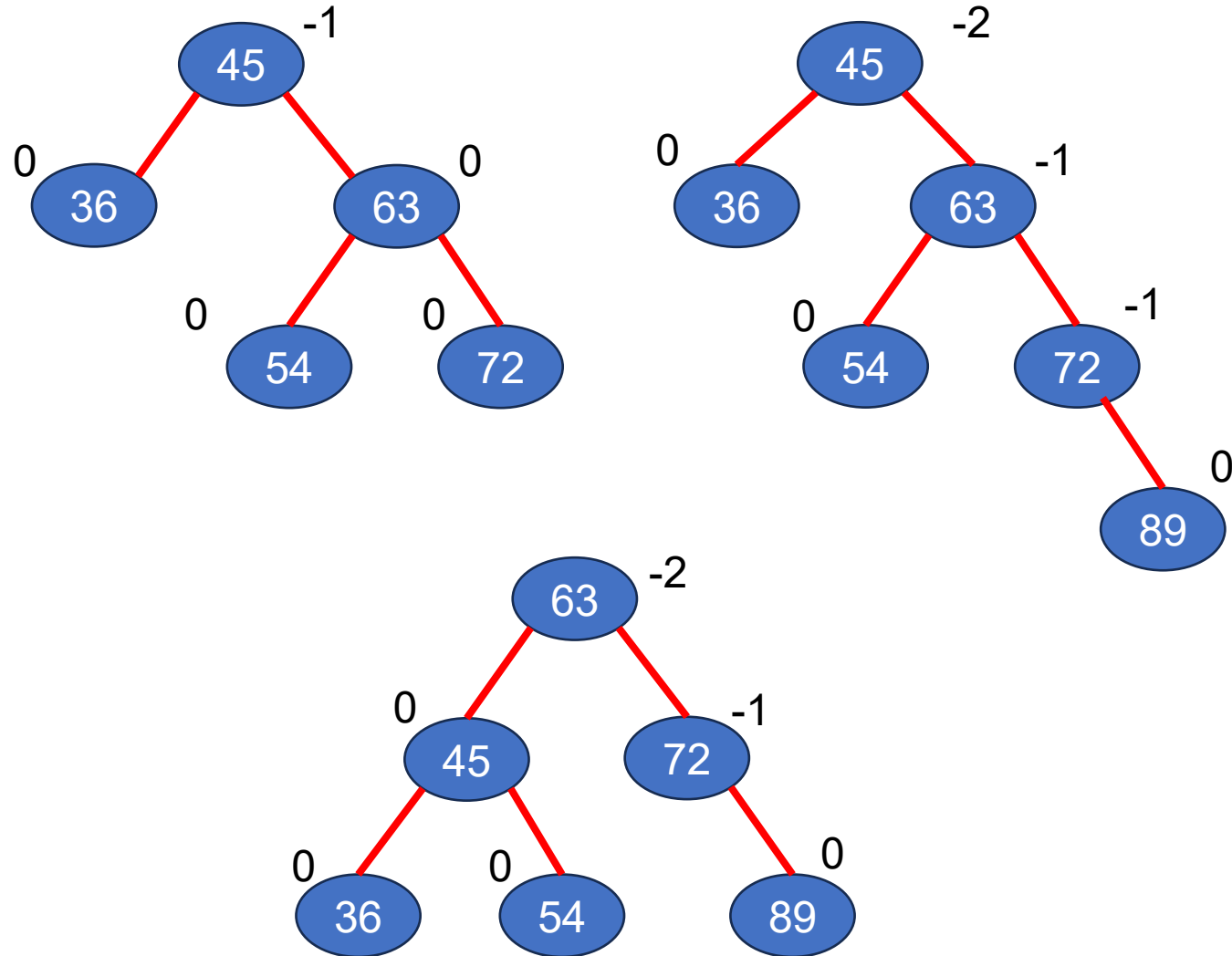


Вращение RR

Дерево (a) является деревом AVL. В дереве (b) новый узел вставляется в правое поддереву правого поддерева критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент баланса которого не равен -1 , 0 или 1), поэтому мы применяем вращение RR, как показано в дереве (c). Обратите внимание, что новый узел теперь стал частью дерева T3. Во время вращения узел B становится корнем, а A и T3 — его левым и правым потомками. T1 и T2 становятся левым и правым поддеревьями A.

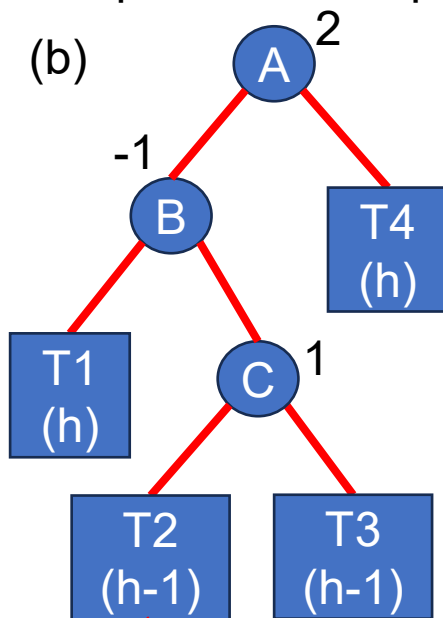
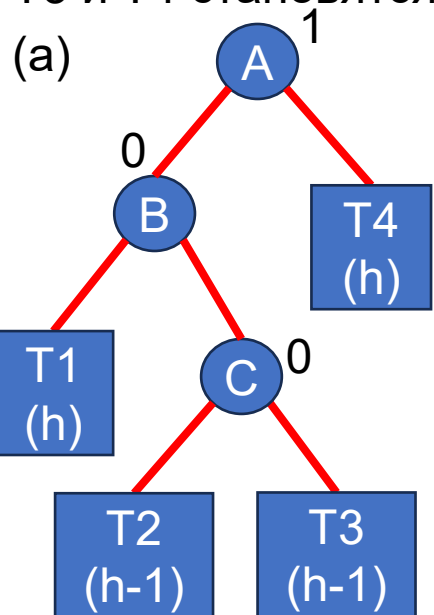


Вращение RR - Пример

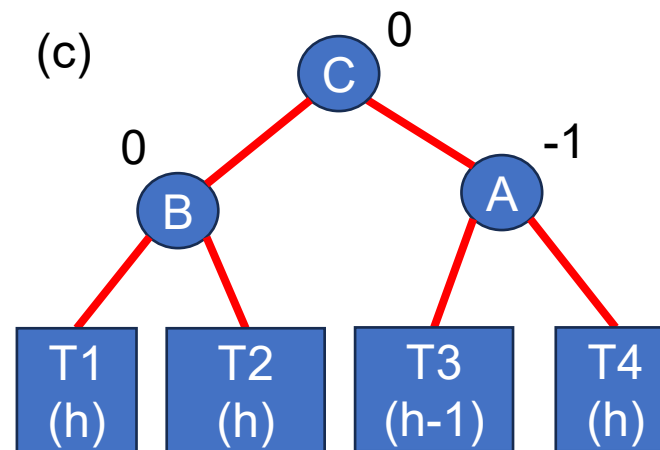


Повороты LR

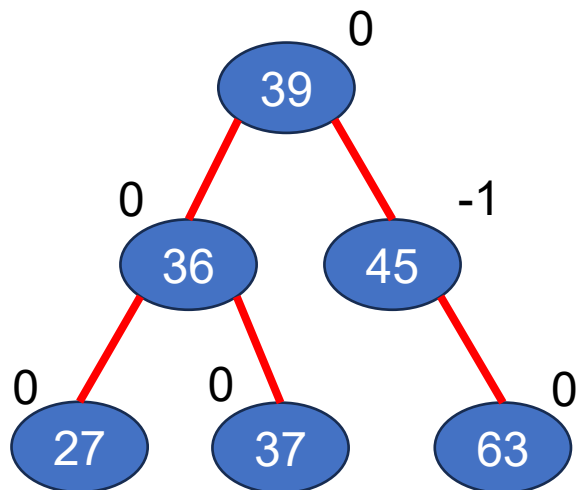
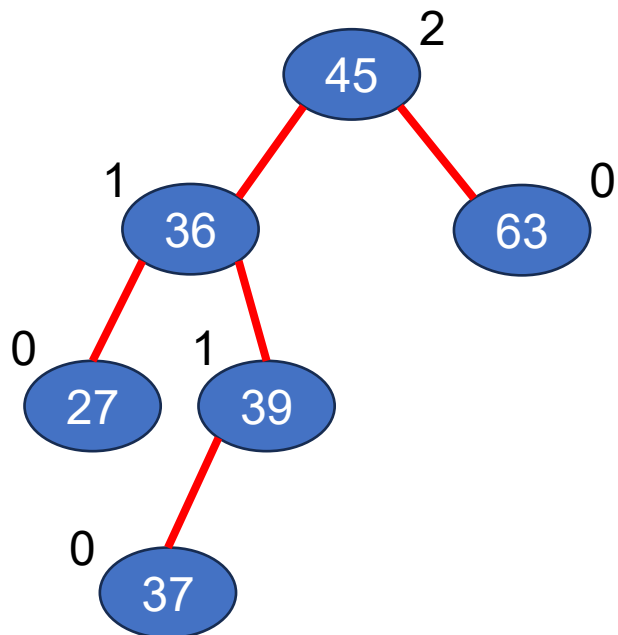
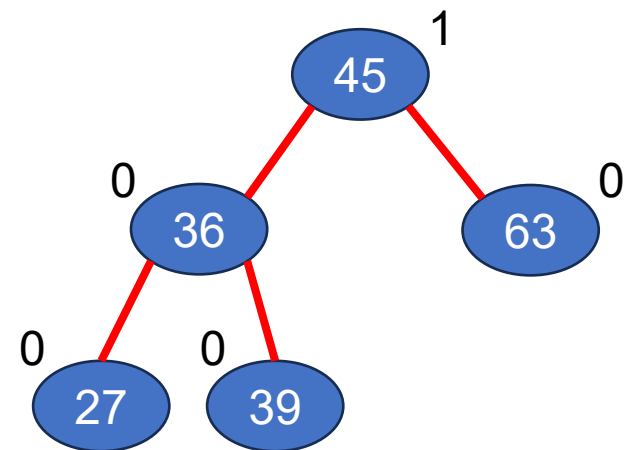
Дерево (a) является деревом AVL. В дереве (b) новый узел вставляется в правое поддереву левого поддерева критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент балансировки которого не равен -1 , 0 или 1), поэтому мы применяем поворот LR, как показано в дереве (c). Обратите внимание, что новый узел теперь стал частью дерева T2. Во время поворота узел C становится корнем, а B и A — его левым и правым потомками. Узел B имеет T1 и T2 в качестве своих левых и правых поддеревьев, а T3 и T4 становятся левым и правым поддеревьями узла A.



New node  Height = h

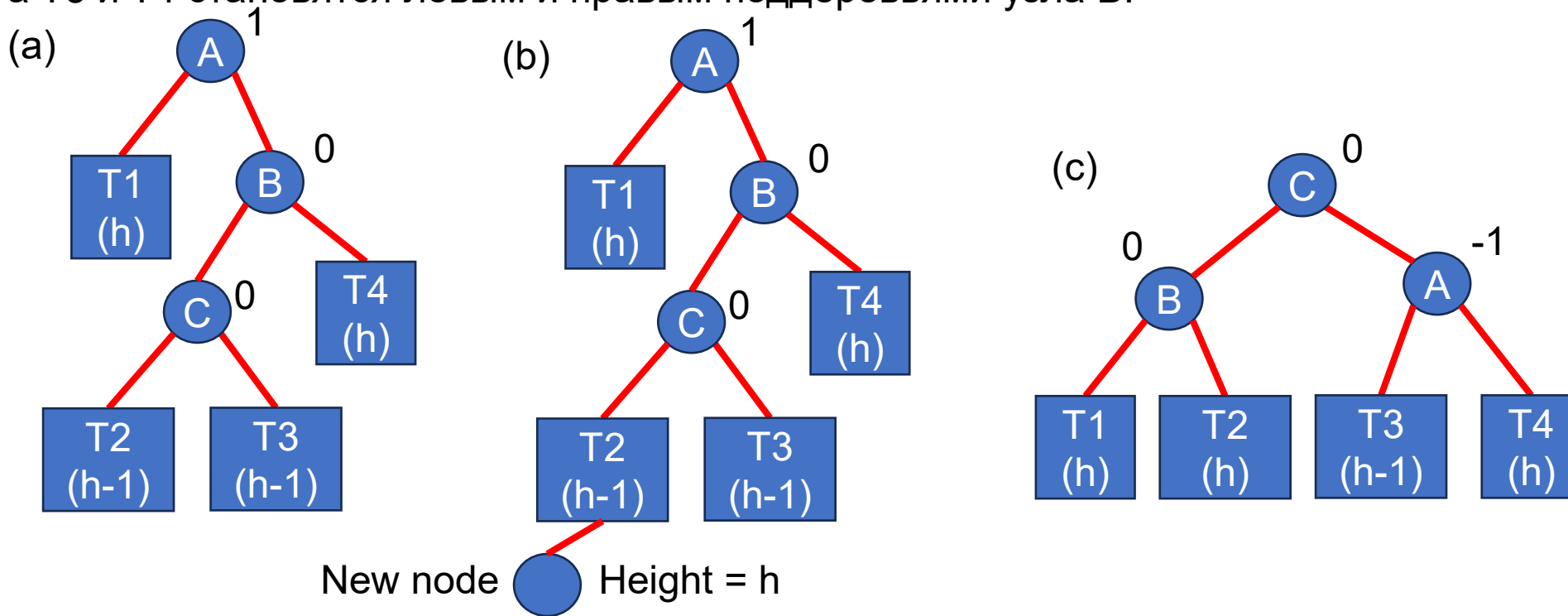


Повороты LR - Пример



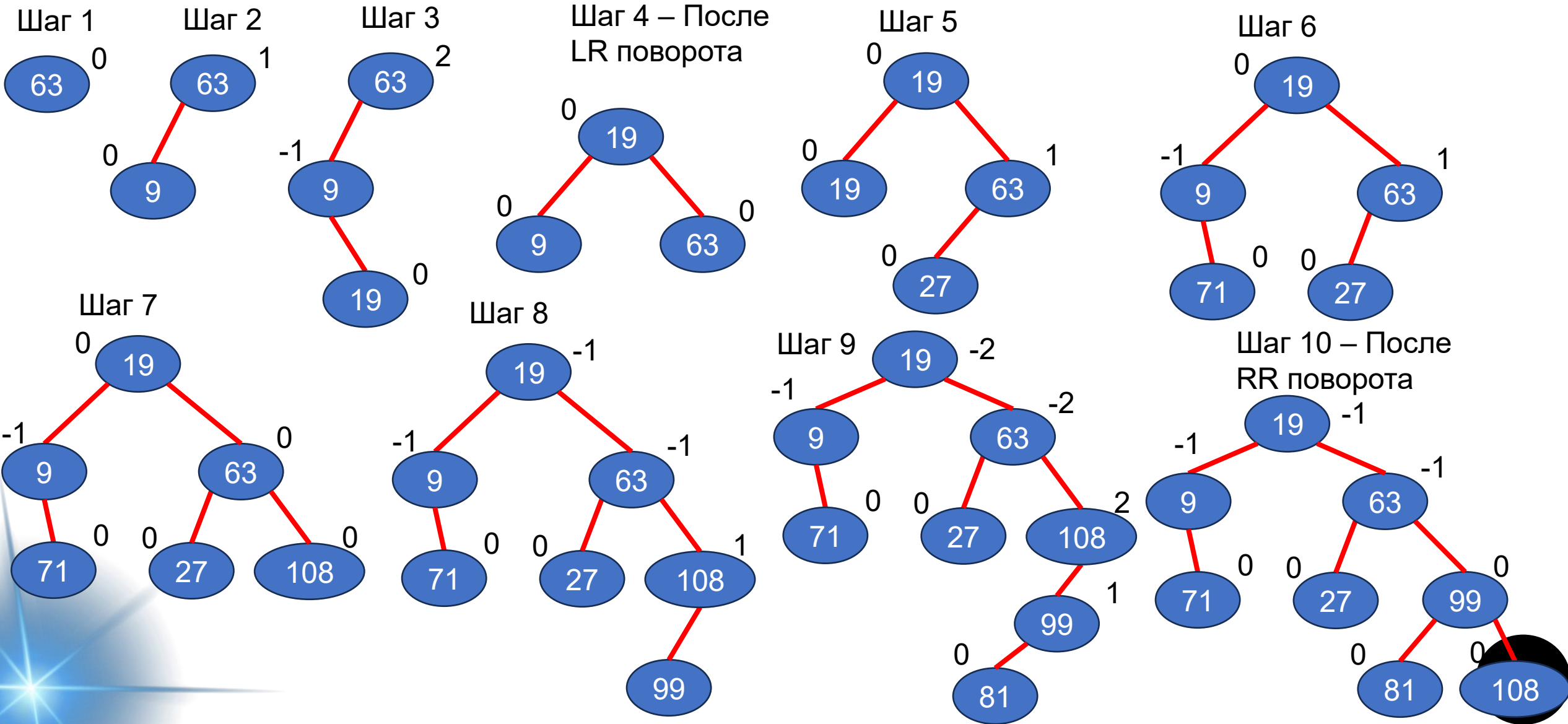
Повороты RL

Дерево (a) является деревом AVL. В дереве (b) новый узел вставляется в левое поддереву правого поддерева критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент балансировки которого не равен -1 , 0 или 1), поэтому мы применяем вращение RL, как показано в дереве (c). Обратите внимание, что новый узел теперь стал частью дерева T2. Во время вращения узел C становится корнем, а A и B — его левым и правым потомками. Узел A имеет T1 и T2 в качестве своих левого и правого поддеревьев, а T3 и T4 становятся левым и правым поддеревьями узла B.



Повороты – общий пример

Построить дерево AVL,
вставив следующие элементы
в указанном порядке.
63, 9, 19, 27, 18, 108, 99, 81.



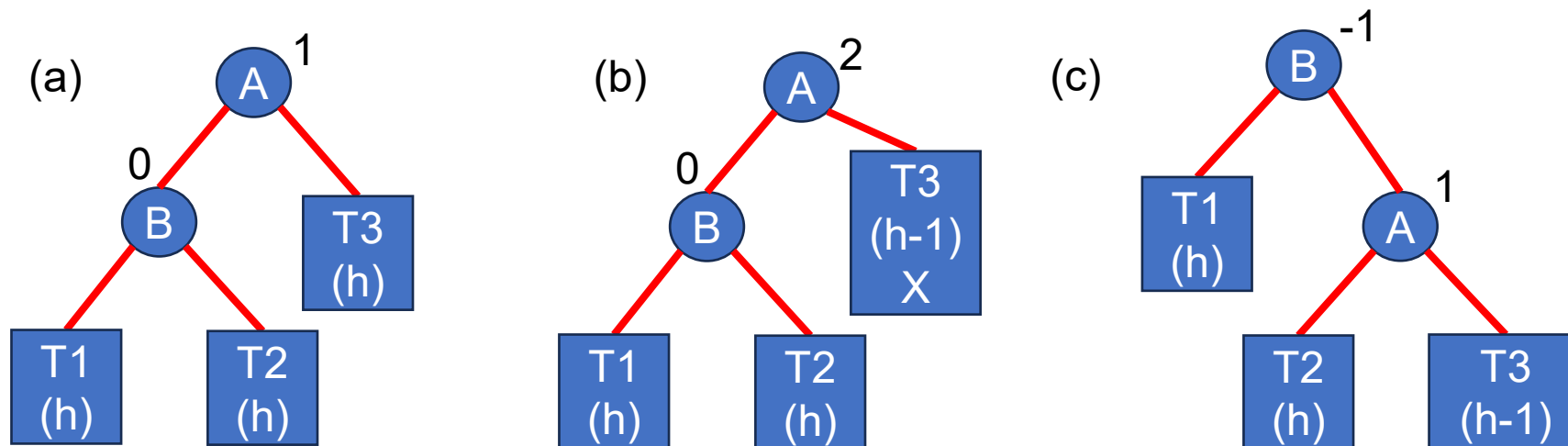
Удаление узла из дерева AVL

Удаление узла в дереве AVL похоже на удаление узла в деревьях бинарного поиска. Но оно идет на шаг впереди. Удаление может нарушить AVL-свойство дерева, поэтому для повторной балансировки дерева AVL нам нужно выполнить вращения. Существует два класса вращений, которые можно выполнить на дереве AVL после удаления заданного узла. Эти вращения — R-вращение и L-вращение. При удалении узла X из дерева AVL, если узел A становится критическим узлом (ближайший узел-предок на пути от X к корневому узлу, который не имеет своего фактора баланса 1, 0 или -1), то тип поворота зависит от того, находится ли X в левом поддереве A или в его правом поддереве. Если удаляемый узел присутствует в левом поддереве A , то применяется поворот L, в противном случае, если X находится в правом поддереве, выполняется поворот R. Кроме того, существует три категории поворотов L и R. Разновидностями поворота L являются повороты L-1, L0 и L1. Соответственно для поворота R существуют повороты R0, R-1 и R1. В этом разделе мы обсудим только поворот R. Повороты L являются зеркальными отражениями поворотов R.

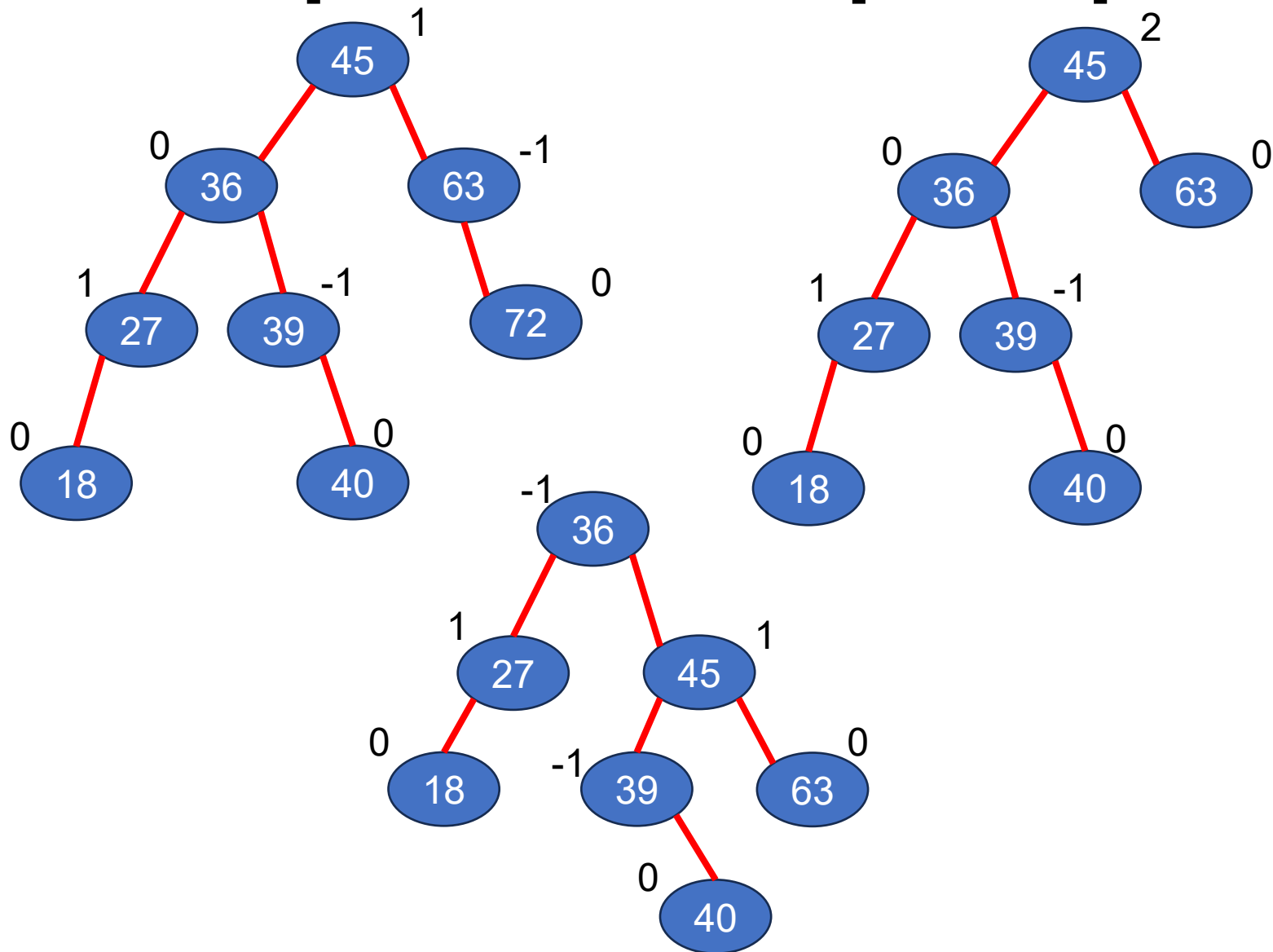


Поворот R0

Дерево (a) является деревом AVL. В дереве (b) узел X должен быть удален из правого поддерева критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент баланса которого не равен -1 , 0 или 1). Поскольку коэффициент баланса узла B равен 0 , мы применяем вращение R0, как показано на дереве (c). В процессе вращения узел B становится корнем, а T1 и A — его левым и правым потомками. T2 и T3 становятся левым и правым поддеревьями A.

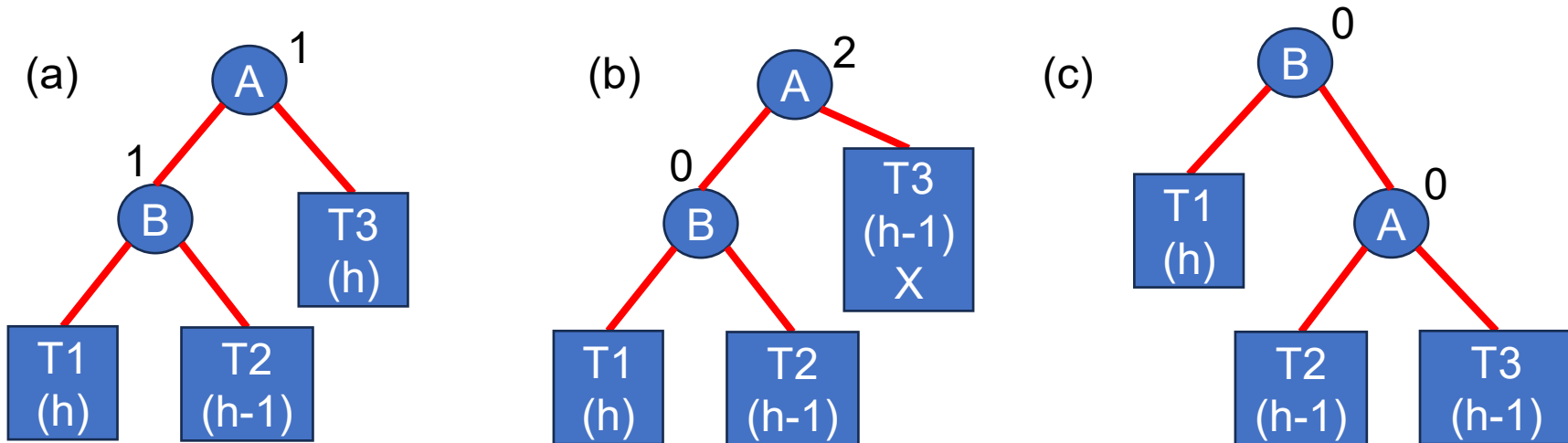


Повороты R0 - Пример

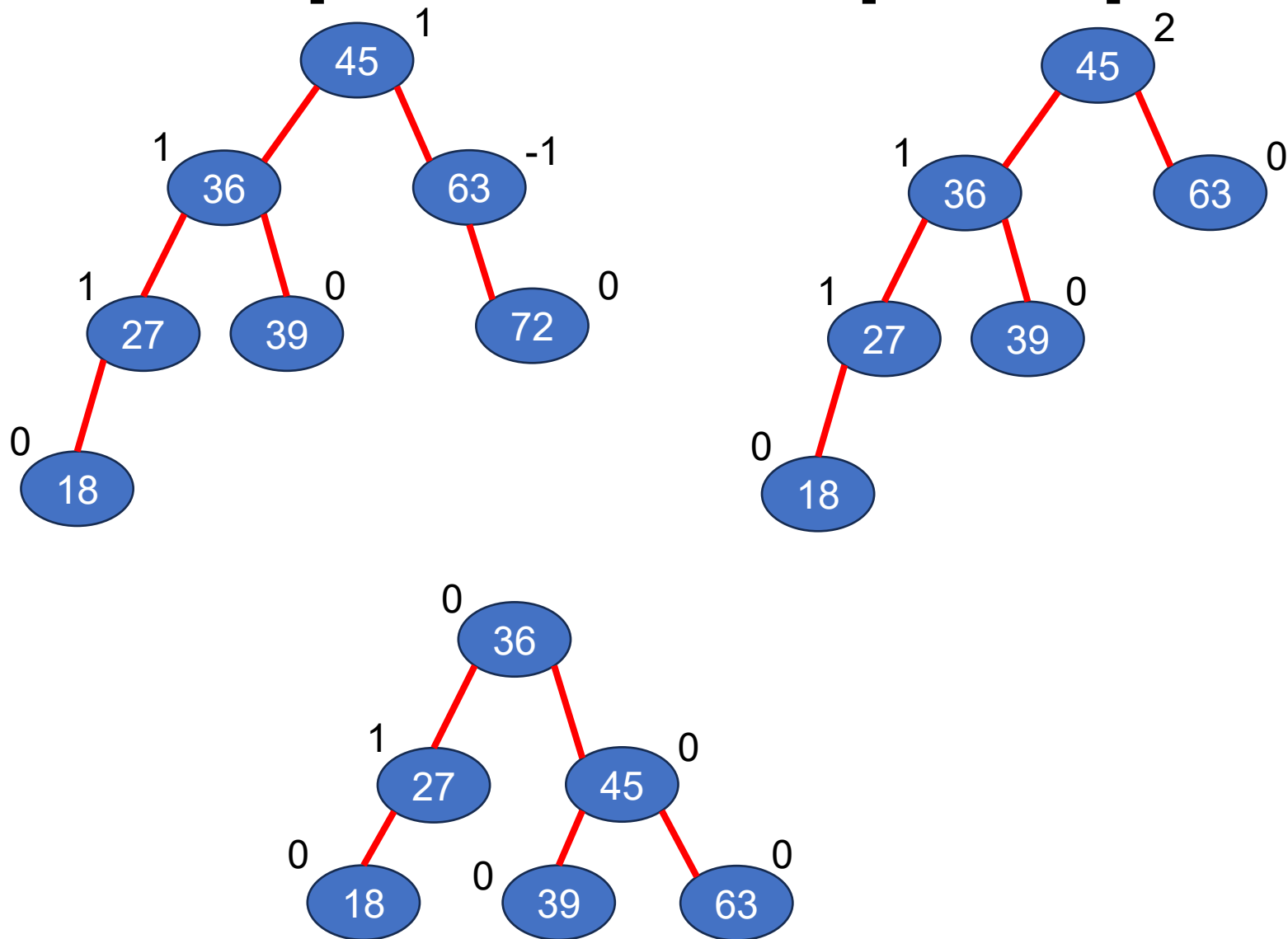


Поворот R1

Пусть В будет корнем левого или правого поддеревья А (критический узел). Вращение R1 применяется, если коэффициент баланса В равен 1. Обратите внимание, что вращения R0 и R1 аналогичны вращениям LL; единственное отличие состоит в том, что вращения R0 и R1 дают разные коэффициенты баланса. Дерево (a) является деревом AVL. В дереве (b) узел X должен быть удален из правого поддерева критического узла А (узел А является критическим узлом, поскольку он является ближайшим предком, коэффициент баланса которого не равен -1 , 0 или 1). Поскольку коэффициент баланса узла В равен 1, мы применяем вращение R1, как показано на дереве (c). В процессе вращения узел В становится корнем, а T1 и А — его левым и правым потомками. T2 и T3 становятся левым и правым поддеревьями А.



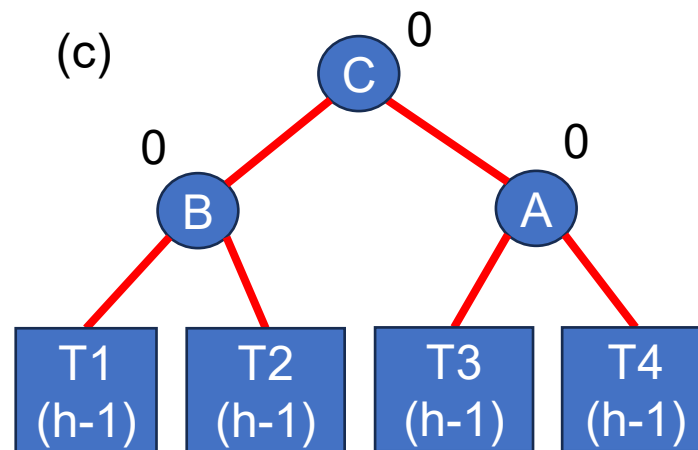
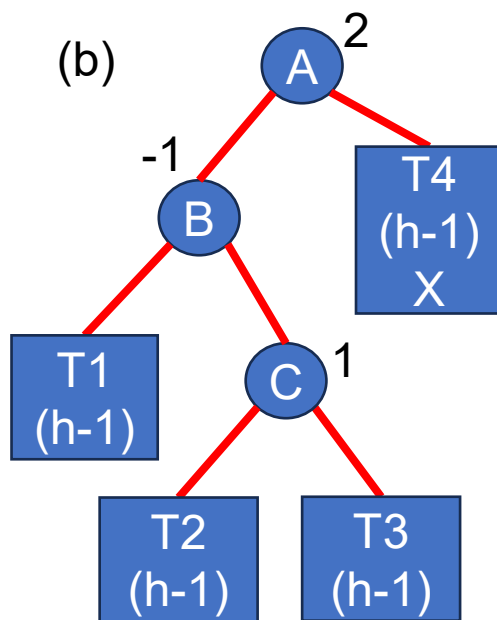
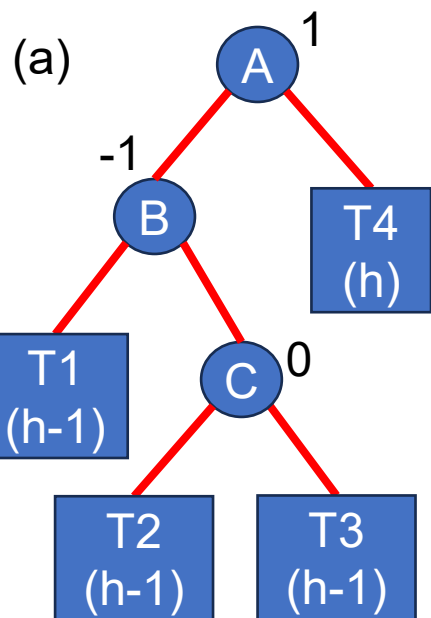
Повороты R1 - Пример



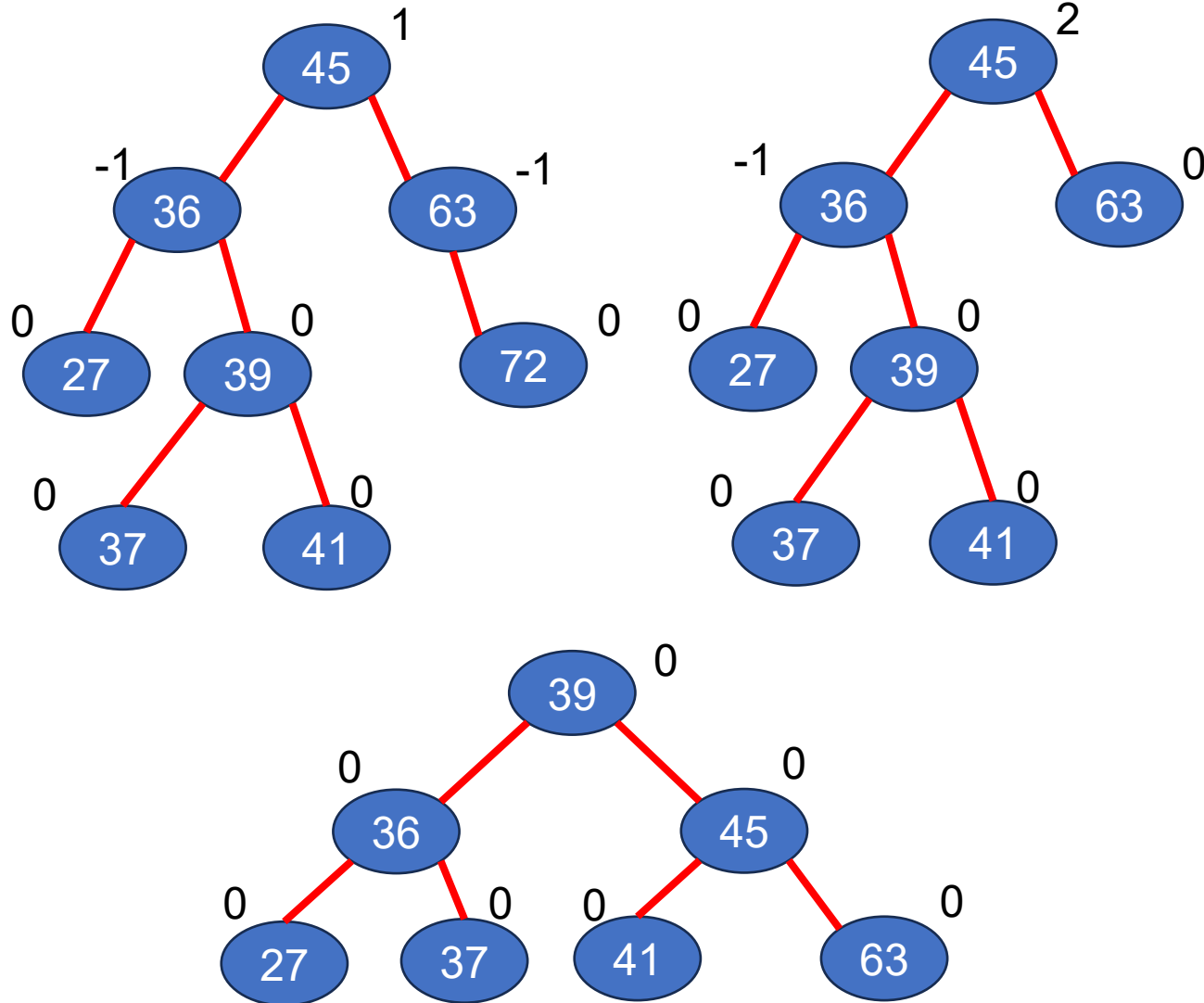
Повороты R-1

Дерево (a) является деревом AVL. В дереве (b) узел X должен быть удален из правого поддеревья критического узла A (узел A является критическим узлом, поскольку он является ближайшим предком, коэффициент баланса которого не равен -1 , 0 или 1). Поскольку коэффициент баланса узла B равен -1 , мы применяем вращение R-1, как показано в дереве (c).

Во время вращения узел C становится корнем, а T1 и A являются его левым и правым потомками. T2 и T3 становятся левым и правым поддеревьями A.



Повороты R-1 - Пример



Красно-черные деревья

Красно-черное дерево — это самобалансирующееся бинарное дерево поиска, которое было изобретено в 1972 году Рудольфом Байером, который назвал его «симметричным бинарным В-деревом». Хотя красно-черное дерево сложное, оно имеет хорошее время выполнения в худшем случае для своих операций и эффективно в использовании, поскольку поиск, вставка и удаление могут быть выполнены за время $O(\log n)$, где n — количество узлов в дереве. Практически, красно-черное дерево — это бинарное дерево поиска, которое вставляет и удаляет разумно, чтобы поддерживать дерево разумно сбалансированным. Особо следует отметить, что в этом дереве в конечных узлах не хранятся никакие данные.



Свойства красно-черных деревьев

Красно-черное дерево — это бинарное дерево поиска, в котором каждый узел имеет цвет, который является либо красным, либо черным. Помимо других ограничений бинарного дерева поиска, красно-черное дерево имеет следующие дополнительные требования:

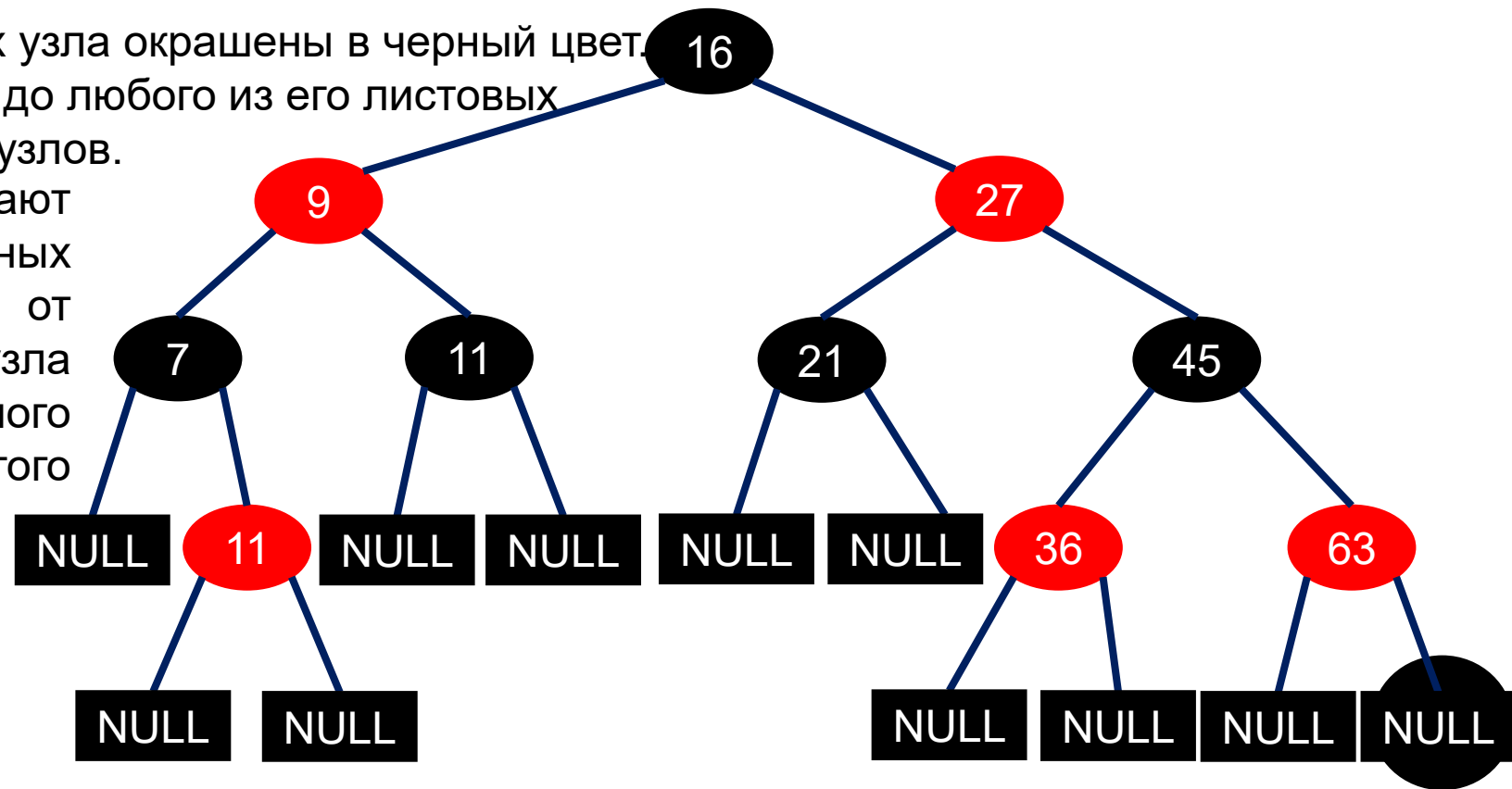
1. Цвет узла может быть либо красным, либо черным.
2. Цвет корневого узла всегда черный.
3. Все листовые узлы черные.
4. У каждого красного узла оба дочерних узла окрашены в черный цвет.
5. Каждый простой путь от данного узла до любого из его листовых узлов имеет равное количество черных узлов.

Эти ограничения обеспечивают критическое свойство красно-черных деревьев. Самый длинный путь от корневого узла до любого листового узла не более чем в два раза длиннее самого короткого пути от корня до любого другого листа в этом дереве.

$$H \leq 2b + O(1)$$

$$n \geq 2^b + 1$$

$$H \leq \log_2 n + 1 + O(1)$$

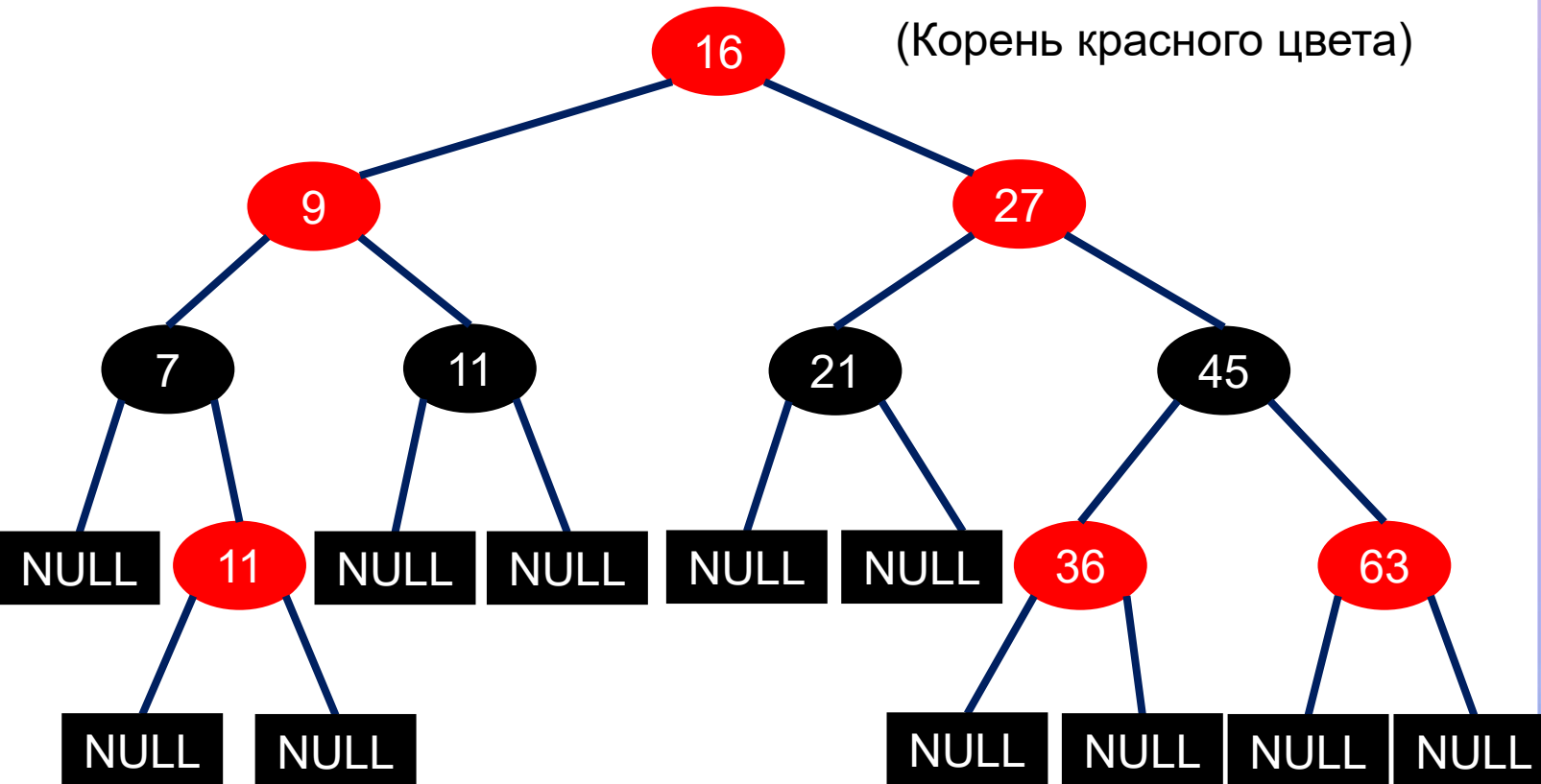


Свойства красно-черных деревьев

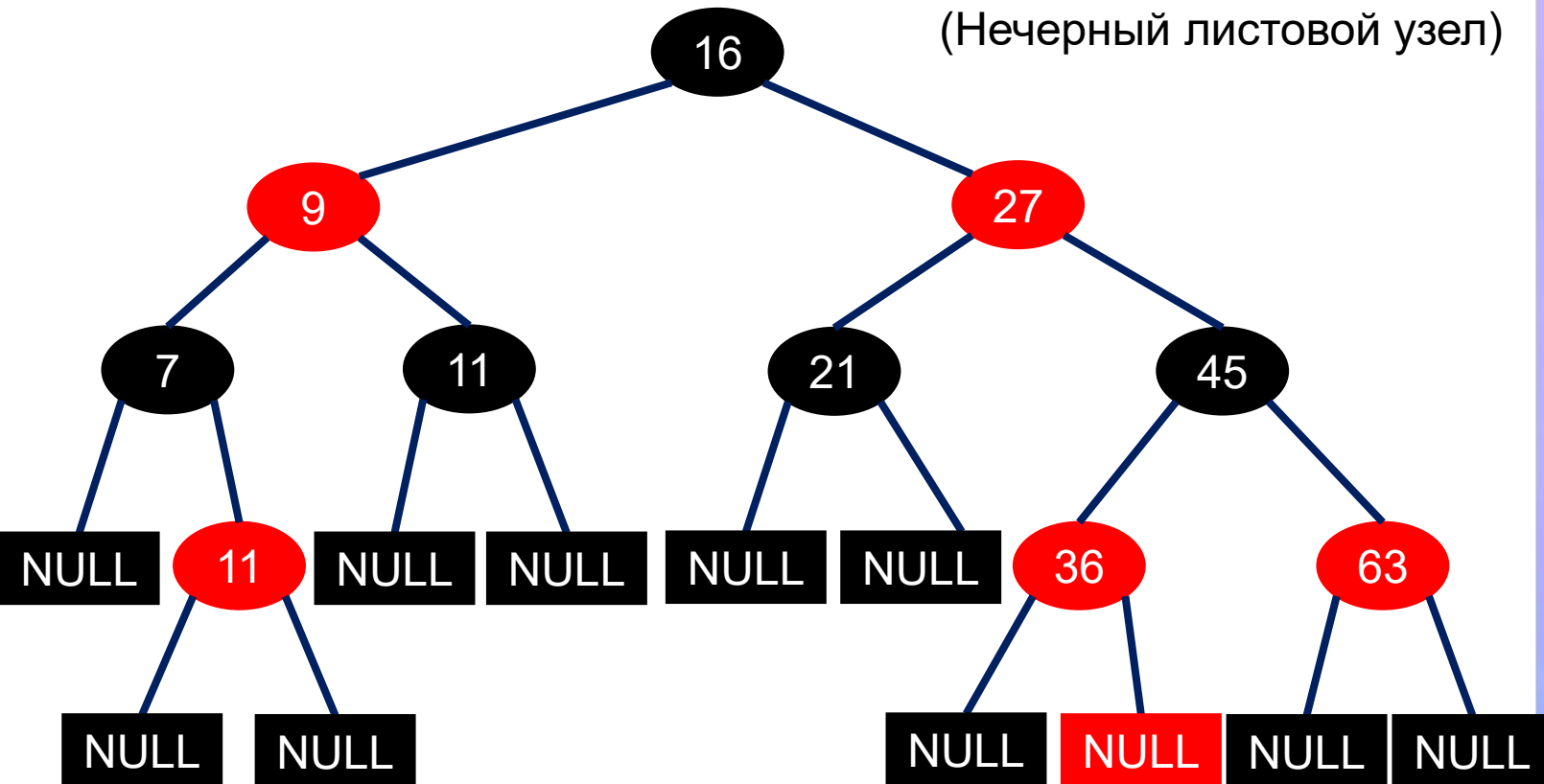
Это приводит к примерно сбалансированному дереву. Поскольку такие операции, как вставка, удаление и поиск, требуют наихудшего времени, пропорционального высоте дерева, эта теоретическая верхняя граница высоты позволяет красно-черным деревьям быть эффективными в наихудшем случае, в отличие от обычных бинарных деревьев поиска. Чтобы понять важность этих свойств, достаточно отметить, что согласно свойству 4 ни один путь не может иметь два красных узла подряд. Самый короткий возможный путь будет иметь все черные узлы, а самый длинный возможный путь будет попеременно иметь красный и черный узел. Поскольку все максимальные пути имеют одинаковое количество черных узлов (свойство 5), ни один путь не будет более чем в два раза длиннее любого другого пути.



Не являются красно-черными деревьями

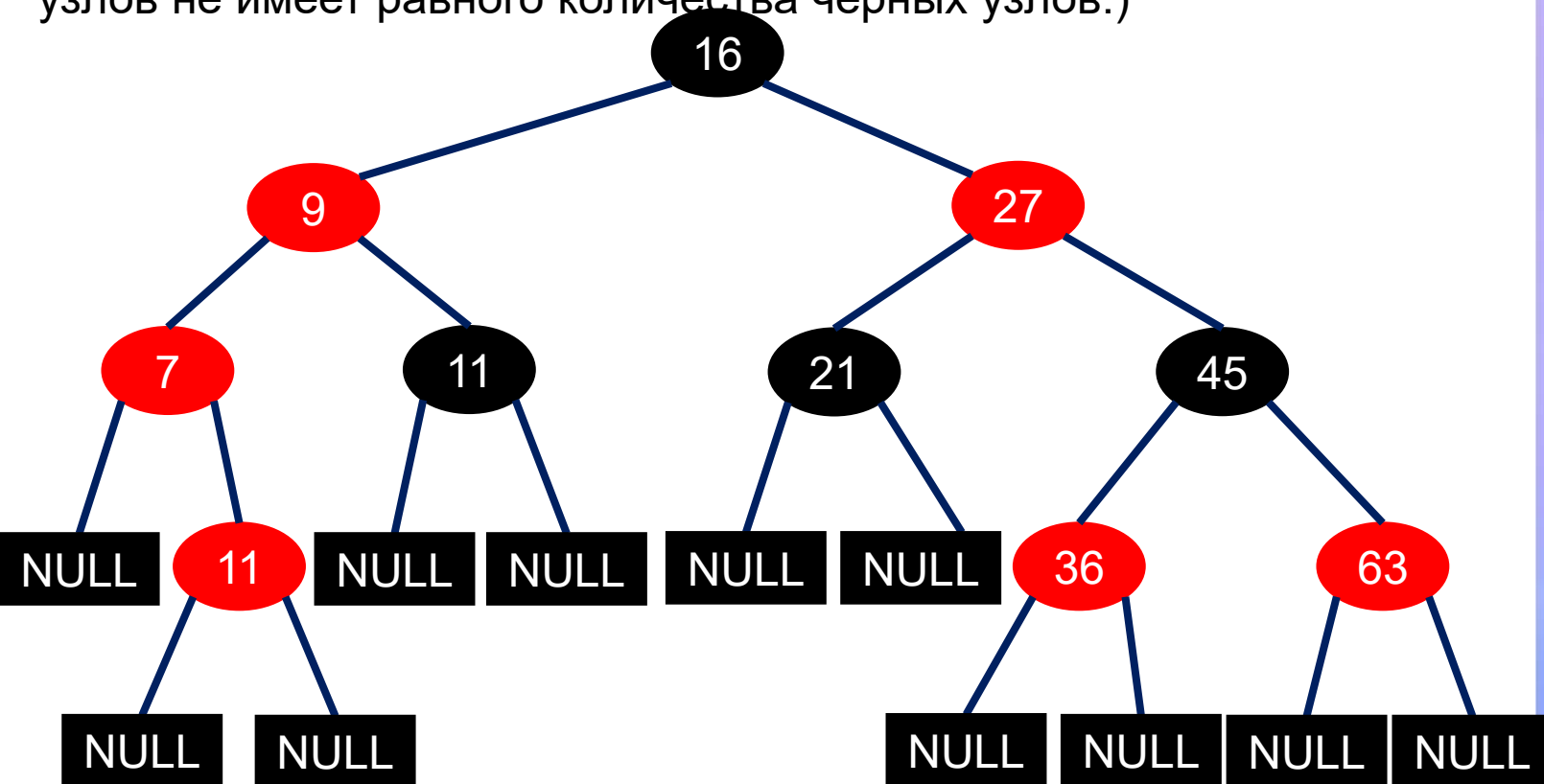


Не являются красно-черными деревьями



Не являются красно-черными деревьями

(Во-первых, каждый красный узел не имеет обоих дочерних узлов, окрашенных в черный цвет. Во-вторых, каждый простой путь от данного узла до любого из его листовых узлов не имеет равного количества черных узлов.)



Операции над красно-черными деревьями

Выполнение операции только для чтения (например, обход узлов в дереве) над красно-черным деревом не требует никаких изменений по сравнению с теми, которые используются для бинарных деревьев поиска. Помните, что каждое красно-черное дерево является частным случаем двоичного дерева поиска. Однако операции вставки и удаления могут нарушать свойства красно-черного дерева. Поэтому эти операции могут создать необходимость восстановления красно-черных свойств, что может потребовать небольшого количества ($O(\log n)$ или амортизированных $O(1)$) изменений цвета.

```
typedef enum
{
    BLACK,
    RED
} bool;
struct node
{
    int val;
    int colour;
    struct node *left;
    struct node *right;
    struct node *parent;
};
```



Вставка узла в красно-черное дерево

Операция вставки начинается так же, как мы добавляем новый узел в двоичное дерево поиска. Однако в двоичном дереве поиска мы всегда добавляем новый узел как лист, в то время как в красно-черном дереве листовые узлы не содержат данных. Поэтому вместо добавления нового узла как листового узла мы добавляем красный внутренний узел, который имеет два черных листовых узла. Обратите внимание, что цвет нового узла красный, а его листовые узлы окрашены в черный цвет.

После добавления нового узла он может нарушить некоторые свойства красно-черного дерева. Поэтому для восстановления их свойства мы проверяем определенные случаи и восстанавливаем свойство в зависимости от случая, который появляется после вставки. Но прежде чем подробно изучать эти случаи, давайте сначала обсудим некоторые важные термины, которые будут использоваться.



Вставка узла в красно-черное дерево

Узел прародителя (G) узла (N) относится к родителю родителя N (P), как в человеческих генеалогических древах. Код C для поиска прародителя узла может быть задан следующим образом:

```
struct node *grand_parent(struct node *n)
{
    // No parent means no grandparent
    if ((n != NULL) && (n->parent != NULL))
        return n->parent->parent;
    else
        return NULL;
}
```



Вставка узла в красно-черное дерево

Узел дяди (U) узла (N) относится к брату родителя N (P), как в человеческих генеалогических древах. Код C для поиска дяди узла может быть задан следующим образом:

```
struct node *uncle(struct node *n)
{
    struct node *g;
    g = grand_parent(n);
    // With no grandparent, there cannot be any uncle
    if (g == NULL)
        return NULL;
    if (n->parent == g->left)
        return g->right;
    else
        return g->left;
}
```



Вставка узла в красно-черное дерево

Когда мы вставляем новый узел в красно-черное дерево, обратите внимание на следующее:

- Все листовые узлы всегда черные. Поэтому свойство 3 всегда выполняется.
- Свойство 4 (оба потомка каждого красного узла черные) находится под угрозой только при добавлении красного узла, перекрашивании черного узла в красный цвет или повороте.
- Свойство 5 (все пути от любого заданного узла до его листовых узлов имеют одинаковое количество черных узлов) находится под угрозой только при добавлении черного узла, перекрашивании красного узла в черный цвет или повороте.



Вставка узла в красно-черное дерево

Случай 1: новый узел N добавляется как корень дерева

В этом случае N перекрашивается в черный цвет, так как корень дерева всегда черный. Поскольку N добавляет один черный узел к каждому пути одновременно, свойство 5 не нарушается. Код C для случая 1 можно задать следующим образом:

```
void case1(struct node *n)
{
    if (n->parent == NULL) // Root node
        n->colour = BLACK;
    else
        case2(n);
}
```



Вставка узла в красно-черное дерево

Случай 2: Родительский узел P нового узла черный

В этом случае оба потомка каждого красного узла черные, поэтому свойство 4 не становится недействительным. Свойство 5 также не находится под угрозой. Это связано с тем, что новый узел N имеет два черных листовых потомка, но поскольку N красный, пути через каждого из его потомков имеют одинаковое количество черных узлов. Код C для проверки для случая 2 можно задать следующим образом:

```
void case2(struct node *n)
{
    if (n->parent->colour == BLACK)
        return;
    /* Свойство красного черного дерева не нарушается */
    else
        case3(n);
}
```

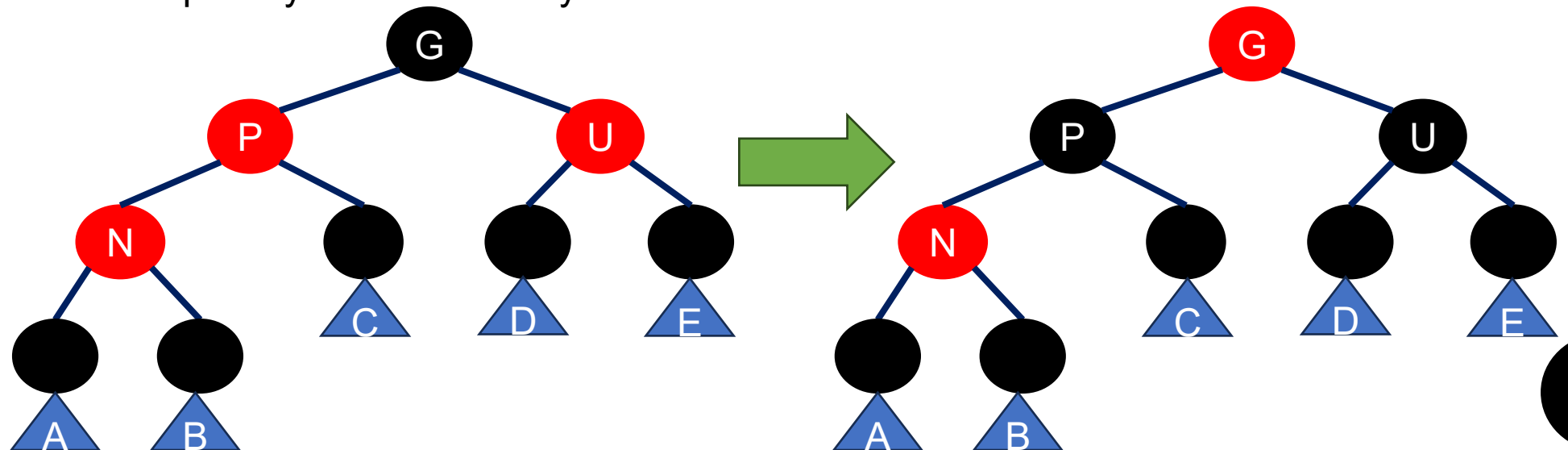


Вставка узла в красно-черное дерево

В следующих случаях предполагается, что N имеет узел-прародителя G , потому что его родитель P красный, а если бы он был корнем, то был бы черным. Таким образом, у N также есть узел-дядя U (независимо от того, является ли U листовым узлом или внутренним узлом).

Случай 3: Если и родитель (P), и дядя (U) красные

В этом случае нарушается Свойство 5, которое гласит, что все пути от любого данного узла до его листовых узлов имеют одинаковое количество черных узлов. Чтобы восстановить Свойство 5, оба узла (P и U) перекрашиваются в черный цвет, а прародитель G перекрашивается в красный цвет. Теперь новый красный узел N имеет черного родителя. Поскольку любой путь через родителя или дядю должен проходить через прародителя, количество черных узлов на этих путях не изменилось.



Вставка узла в красно-черное дерево

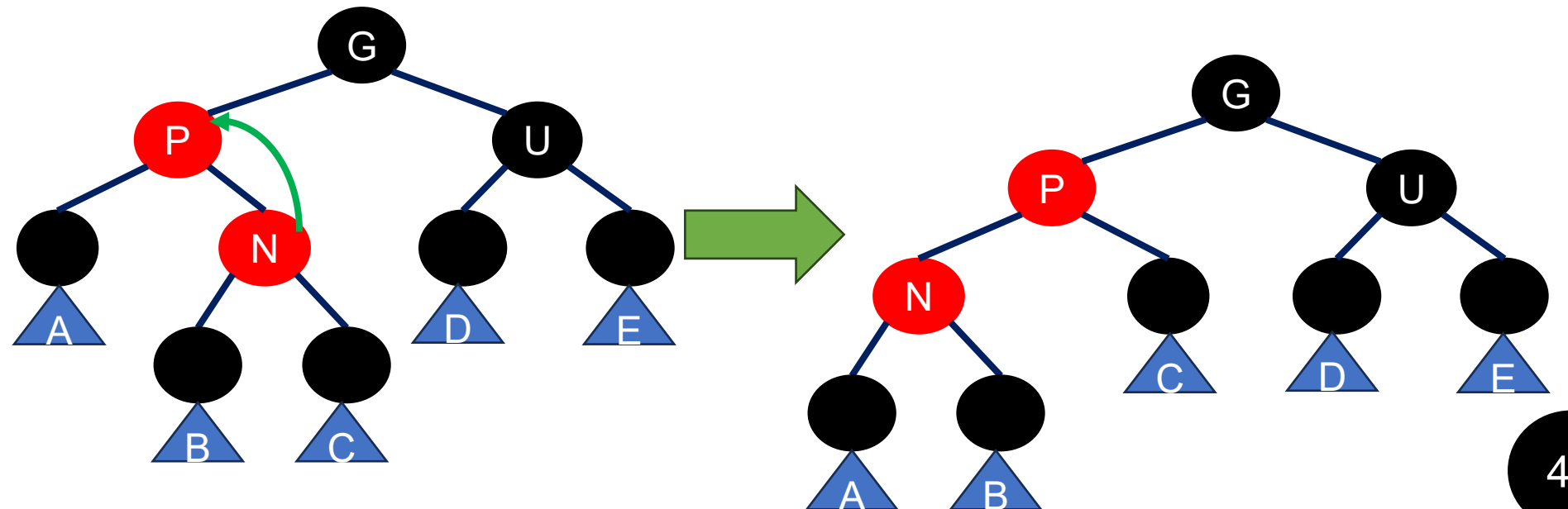
Однако прародитель G теперь может нарушить Свойство 2, которое гласит, что корневой узел всегда черный, или Свойство 4, которое гласит, что оба потомка каждого красного узла черные. Свойство 4 будет нарушено, когда у G есть красный родитель. Чтобы исправить эту проблему, вся эта процедура рекурсивно выполняется на G из случая 1. Код C для вставки случая 3 выглядит следующим образом:

```
void case3(struct node *n)
{
    struct node *u, *g;
    u = uncle(n);
    g = grand_parent(n);
    if ((u != NULL) && (u->colour == RED))
    {
        n->parent->colour = BLACK;
        u->colour = BLACK;
        g->colour = RED;
        case1(g);
    }
    else
        insert_case4(n);
}
```

Вставка узла в красно-черное дерево

Случай 4: Родитель P — красный, но дядя U — черный, а N — правый потомок P , а P — левый потомок G

Чтобы исправить эту проблему, выполняется поворот влево, чтобы поменять роли нового узла N и его родителя P . После поворота обратите внимание, что в коде C мы переименовали N и P , а затем вызывается случай 5 для работы с родителем нового узла. Это делается потому, что свойство 4, которое гласит, что оба потомка каждого красного узла должны быть черными, все еще нарушается. На рисунке показана вставка случая 4. Обратите внимание, что в случае, если N — левый потомок P , а P — правый потомок G , нам нужно выполнить поворот вправо. В коде C , который обрабатывает случай 4, мы проверяем P и N , а затем выполняем поворот влево или вправо.



Вставка узла в красно-черное дерево

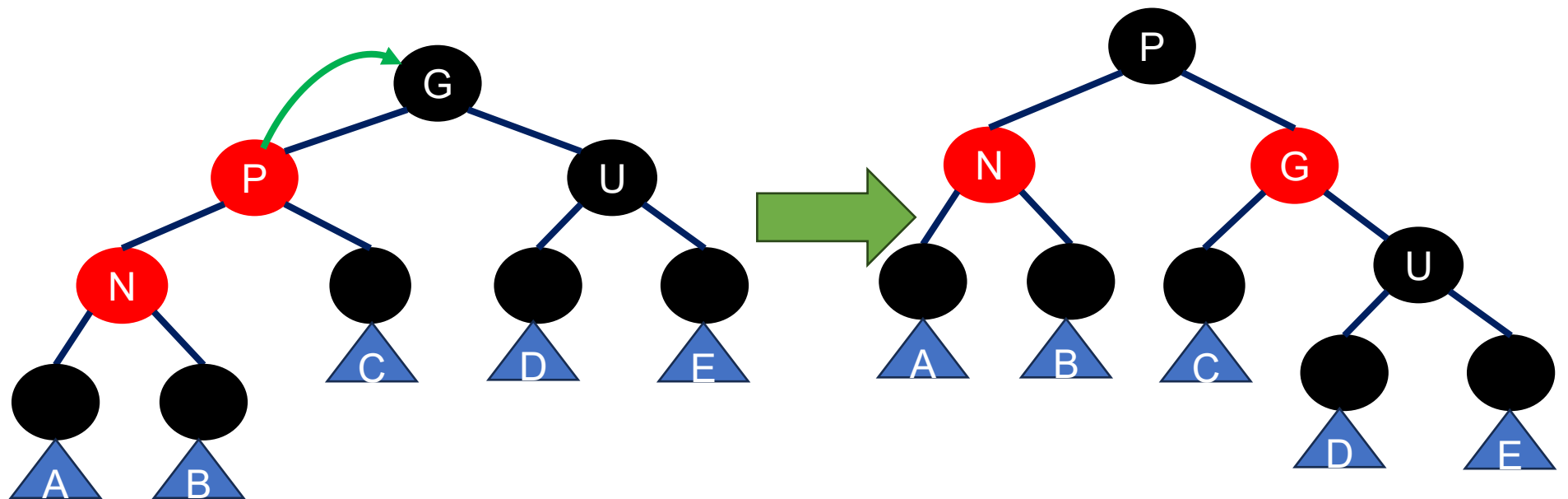
Однако прародитель G теперь может нарушить Свойство 2, которое гласит, что корневой узел всегда черный, или Свойство 4, которое гласит, что оба потомка каждого красного узла черные. Свойство 4 будет нарушено, когда у G есть красный родитель. Чтобы исправить эту проблему, вся эта процедура рекурсивно выполняется на G из случая 1. Код C для вставки случая 3 выглядит следующим образом:

```
void case4(struct node *n)
{
    struct node *g = grand_parent(n);
    if ((n == n->parent->right) && (n->parent == g->left))
    {
        rotate_left(n->parent);
        n = n->left;
    }
    else if ((n == n->parent->left) && (n->parent == g->right))
    {
        rotate_right(n->parent);
        n = n->right;
    }
    case5(n);
}
```

Вставка узла в красно-черное дерево

Случай 5: Родительский узел P — красный, но дядя U — черный, а новый узел N — левый потомок P , а P — левый потомок своего родителя G .

Чтобы исправить эту проблему, выполняется поворот вправо на G (прародительском узле N). После этого поворота бывший родитель P теперь является родителем как нового узла N , так и бывшего прародителя G . Мы знаем, что цвет G черный (потому что в противном случае его бывший потомок P не мог бы быть красным), поэтому теперь поменяем цвета P и G так, чтобы полученное дерево удовлетворяло Свойству 4, которое гласит, что оба потомка красного узла являются черными. Случай 5 вставки проиллюстрирован на рисунке.



Вставка узла в красно-черное дерево

Случай 5: Родительский узел P — красный, но дядя U — черный, а новый узел N — левый потомок P , а P — левый потомок своего родителя G .

```
void case5(struct node *n)
{
    struct node *g;
    g = grandparent(n);
    if ((n == n->parent->left) && (n->parent == g->left))
        rotate_right(g);
    else if ((n == n->parent->right) && (n->parent == g->right))
        rotate_left(g);
    n->parent->colour = BLACK;
    g->colour = RED;
}
```


Удаление узла из красно-черного дерева

Мы начинаем удалять узел из красно-черного дерева так же, как и в случае с бинарным деревом поиска. В бинарном дереве поиска, когда мы удаляем узел с двумя нелистовыми дочерними элементами, мы находим либо максимальный элемент в его левом поддереве узла, либо минимальный элемент в его правом поддереве и перемещаем его значение в удаляемый узел. После этого мы удаляем узел, из которого мы скопировали значение. Обратите внимание, что этот узел должен иметь менее двух нелистовых дочерних элементов. Поэтому простое копирование значения не нарушает никаких красно-черных свойств, а просто сводит проблему удаления к проблеме удаления узла с не более чем одним нелистовым дочерним элементом.

Предположим, что удаляем узел с не более чем одним нелистовым дочерним элементом, который мы назовем его дочерним элементом. Если у этого узла есть оба листовых дочерних элемента, то пусть один из них будет его дочерним элементом.

При удалении узла, если его цвет красный, то мы можем просто заменить его дочерним элементом, который должен быть черным. Все пути через удаленный узел будут просто проходить через один красный узел меньше, и как родительский, так и дочерний узел удаленного узла должны быть черными, поэтому ни одно из свойств не будет нарушено.



Удаление узла из красно-черного дерева

Другой простой случай — когда мы удаляем черный узел, у которого есть красный дочерний узел. В этом случае могут быть нарушены свойства 4 и 5, поэтому для их восстановления просто перекрасьте дочерний узел удаленного узла в черный цвет. Однако возникает сложная ситуация, когда и удаляемый узел, и его дочерний узел черные. В этом случае мы начинаем с замены удаляемого узла его дочерним узлом. В коде C мы обозначаем дочерний узел как (в его новой позиции) N, а его родственный узел (другой дочерний узел его нового родителя) как S. Код C для поиска родственного узла можно задать следующим образом:

```
struct node *sibling(struct node *n)
{
    if (n == n->parent->left)
        return n->parent->right;
    else
        return n->parent->left;
}
```



Удаление узла из красно-черного дерева

Мы можем начать процесс удаления, используя следующий код, где функция `replace_node` подставляет дочерний узел на место `N` в дереве. Для удобства мы предполагаем, что пустые листья представлены реальными объектами узлов, а не `NULL`. Когда и `N`, и его родительский узел `P` черные, то удаление `P` приведет к тому, что пути, которые предшествуют `N`, будут иметь на один черный узел меньше, чем другие пути. Это нарушит Свойство 5. Поэтому дерево необходимо перебалансировать. Необходимо рассмотреть несколько случаев

```
void delete_child(struct node *n)
{
    /* If N has at most one non-null child */
    struct node *child;
    if (is_leaf(n->right))
        child = n->left;
    else
        child = n->right;
    replace_node(n, child);
    if (n->colour == BLACK)
    {
        if (child->colour == RED)
            child->colour = BLACK;
        else
            del_case1(child);
    }
    free(n);
}
```



Удаление узла из красно-черного дерева

Случай 1: N — *новый корень*

В этом случае мы удалили один черный узел из каждого пути, и новый корень черный, поэтому ни одно из свойств не нарушено.

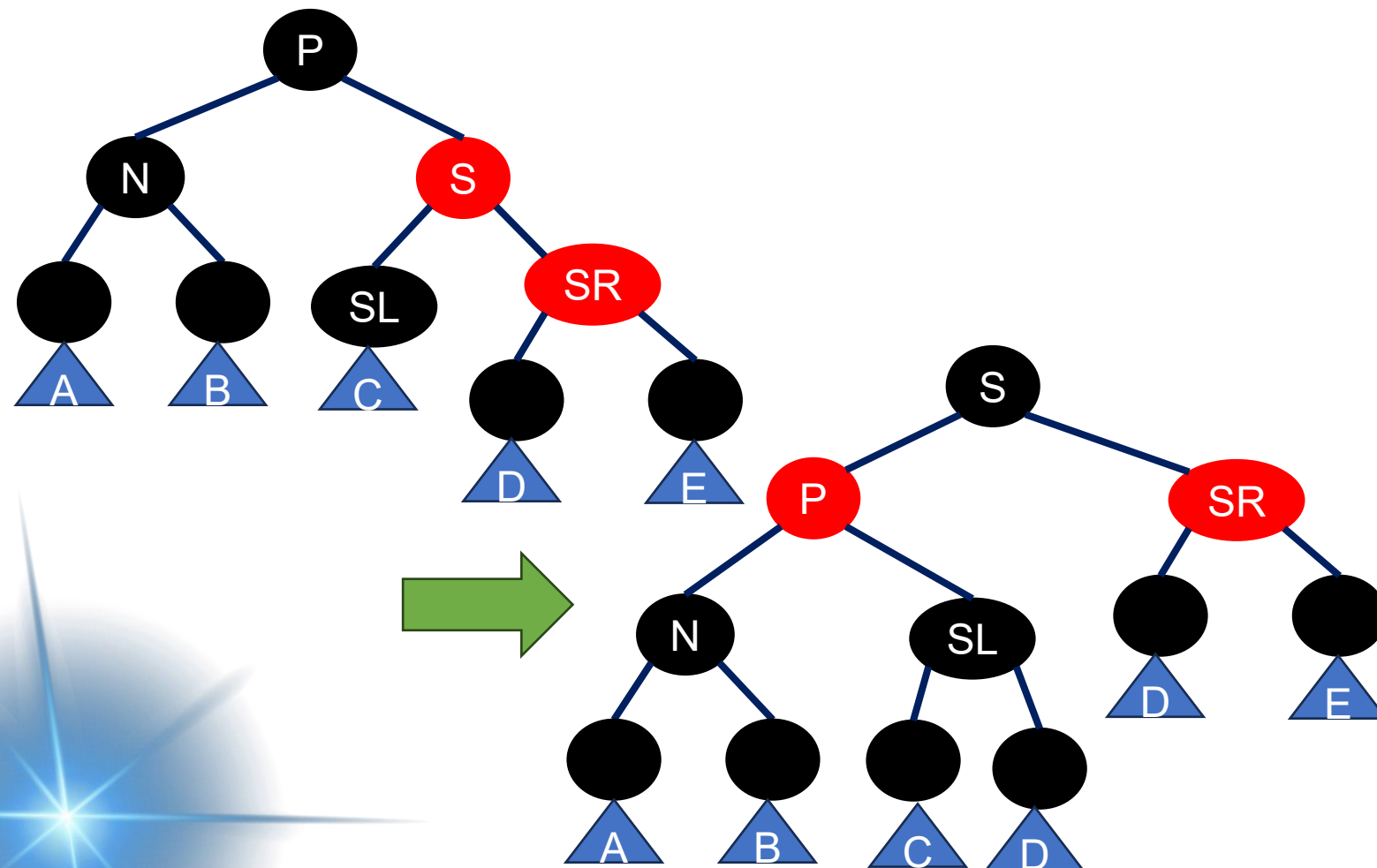
```
void del_case1(struct node *n)
{
    if (n->parent != NULL)
        del_case2(n);
}
```



Удаление узла из красно-черного дерева

Случай 2: родственный элемент S — красный

В этом случае поменяйте цвета P и S, а затем поверните влево в P. В полученном дереве S станет прародителем N.

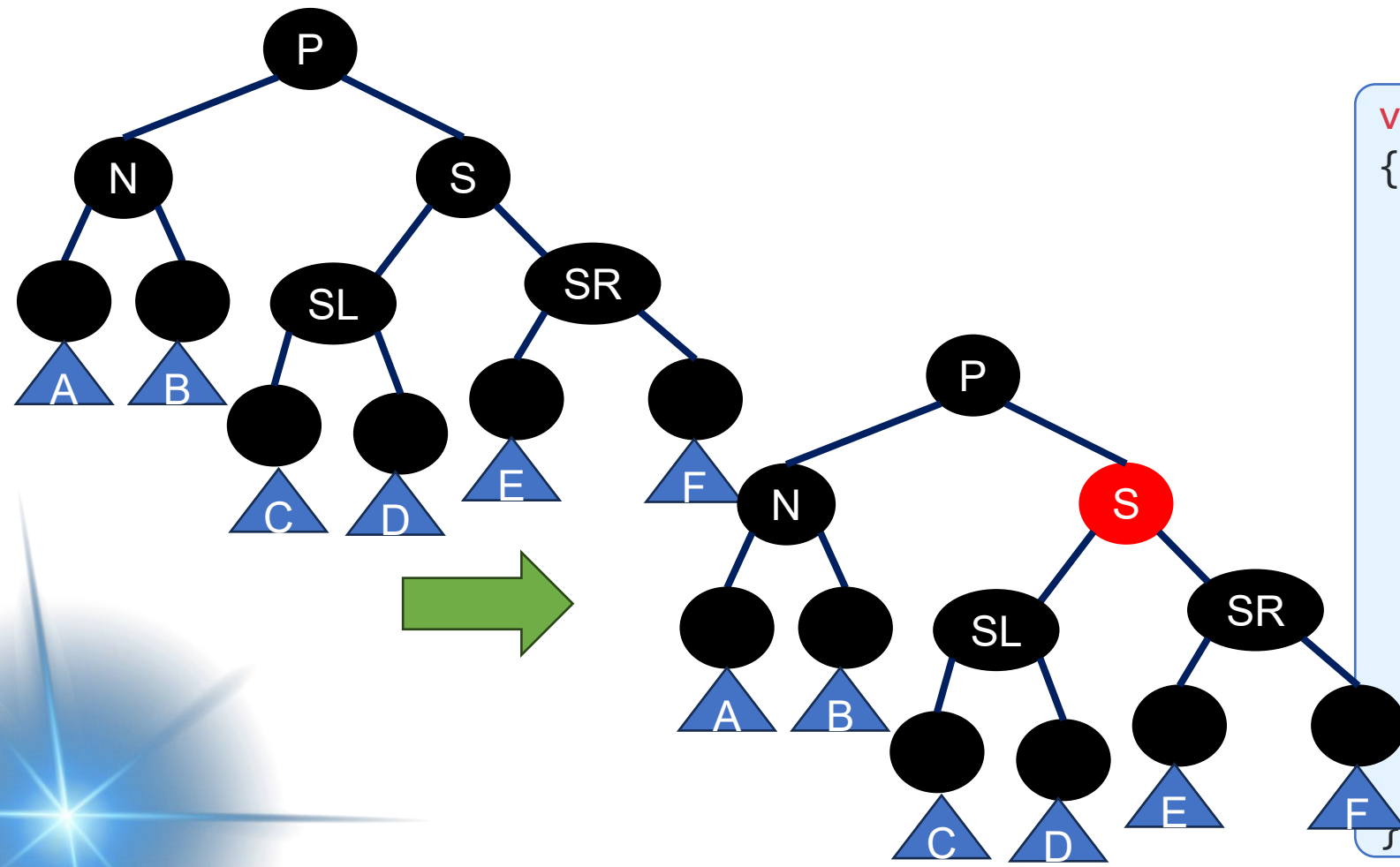


```
void del_case2(struct node *n)
{
    struct node *s;
    s = sibling(n);
    if (s->colour == RED)
    {
        if (n == n->parent->left)
            rotate_left(n->parent);
        else
            rotate_right(n->parent);
        n->parent->colour = RED;
        s->colour = BLACK;
    }
    del_case3(n);
}
```


Удаление узла из красно-черного дерева

Случай 3: P , S и потомки S черные

В этом случае просто перекрасьте S в красный цвет. В полученном дереве все пути, проходящие через S , будут иметь на один черный узел меньше.

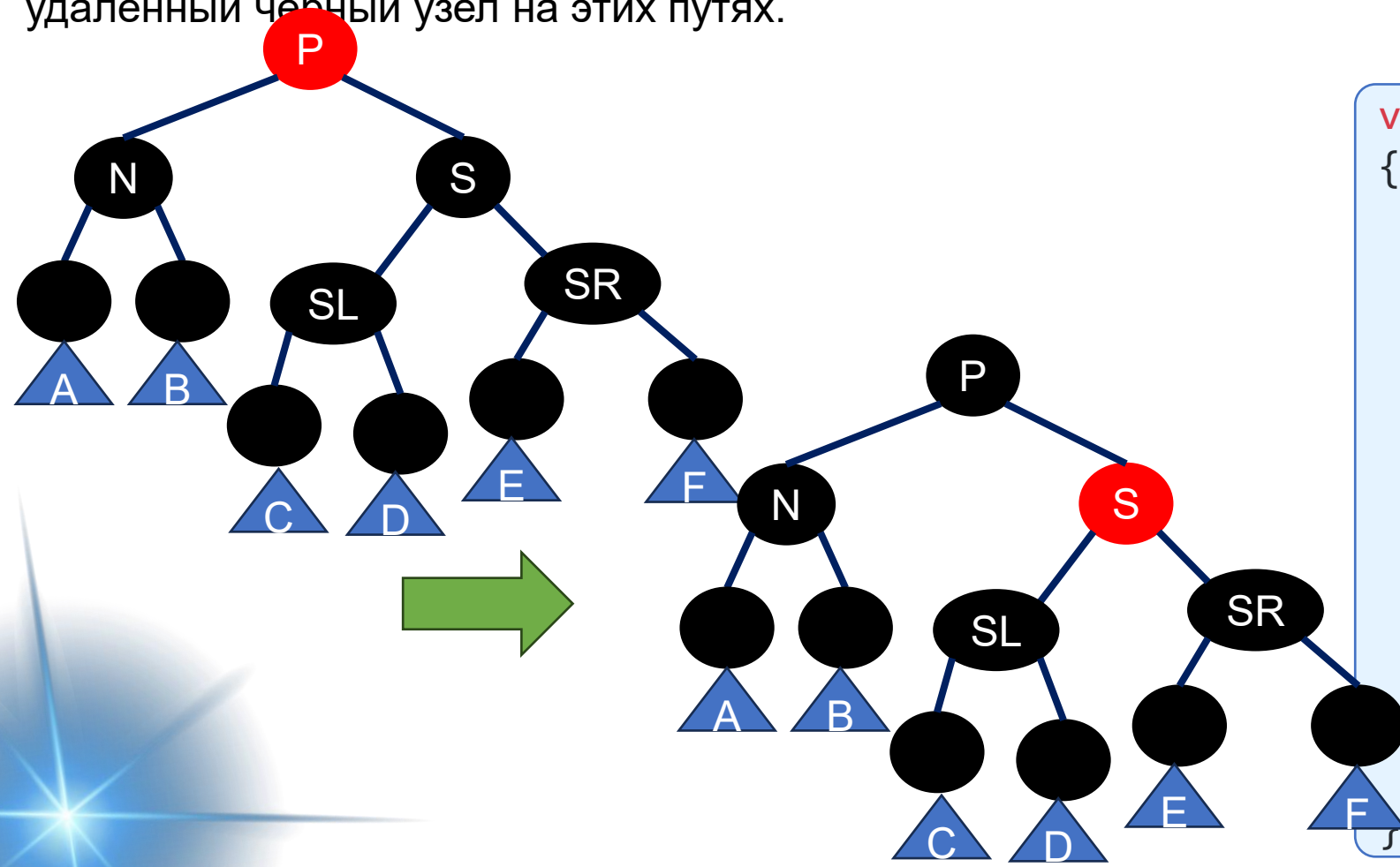


```
void del_case3(struct node *n)
{
    struct node *s;
    s = sibling(n);
    if ((n->parent->colour == BLACK)
        && (s->colour == BLACK) &&
        (s->left->colour == BLACK) &&
        (s->right->colour == BLACK))
    {
        s->colour = RED;
        del_case1(n->parent);
    }
    else
        del_case4(n);
}
```

Удаление узла из красно-черного дерева

Случай 4: Дочерние элементы N и S черные, а P красный

В этом случае мы меняем цвета S и P . Хотя это не повлияет на количество черных узлов на путях, проходящих через S , это добавит один черный узел к путям, проходящим через N , компенсируя удаленный черный узел на этих путях.

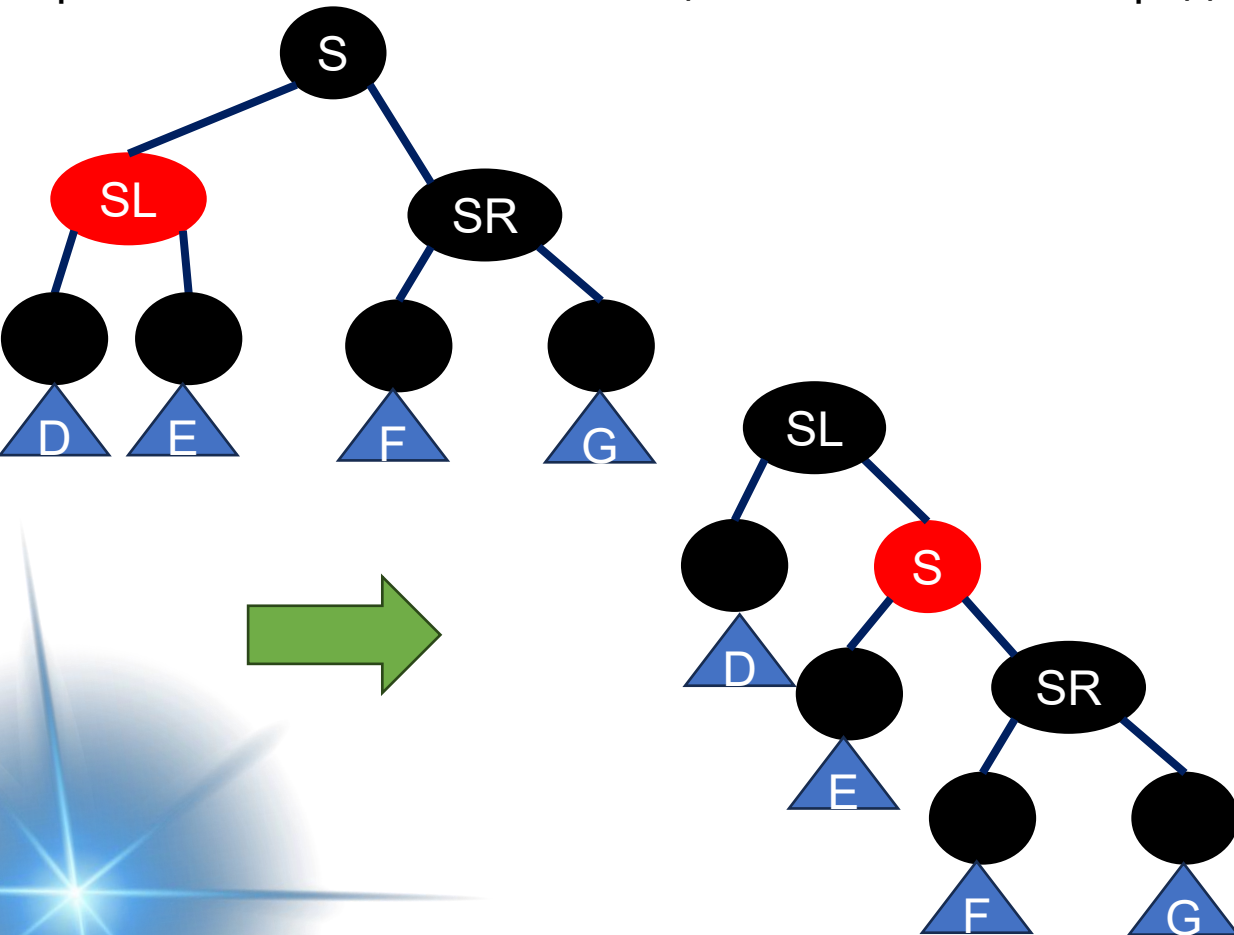


```
void del_case4(struct node *n)
{
    struct node *s;
    s = sibling(n);
    if ((n->parent->colour == RED)
        && (s->colour == BLACK) &&
        (s->left->colour == BLACK) &&
        (s->right->colour == BLACK))
    {
        s->colour = RED;
        n->parent->colour = BLACK;
    }
    else
        del_case5(n);
}
```

Удаление узла из красно-черного дерева

Случай 5: N — левый потомок P , а S — черный, левый потомок S — красный, правый потомок S — черный.

В этом случае выполните поворот вправо в S . После поворота левый потомок S станет родителем S и новым братом N . Также поменяйте цвета S и его нового родителя.

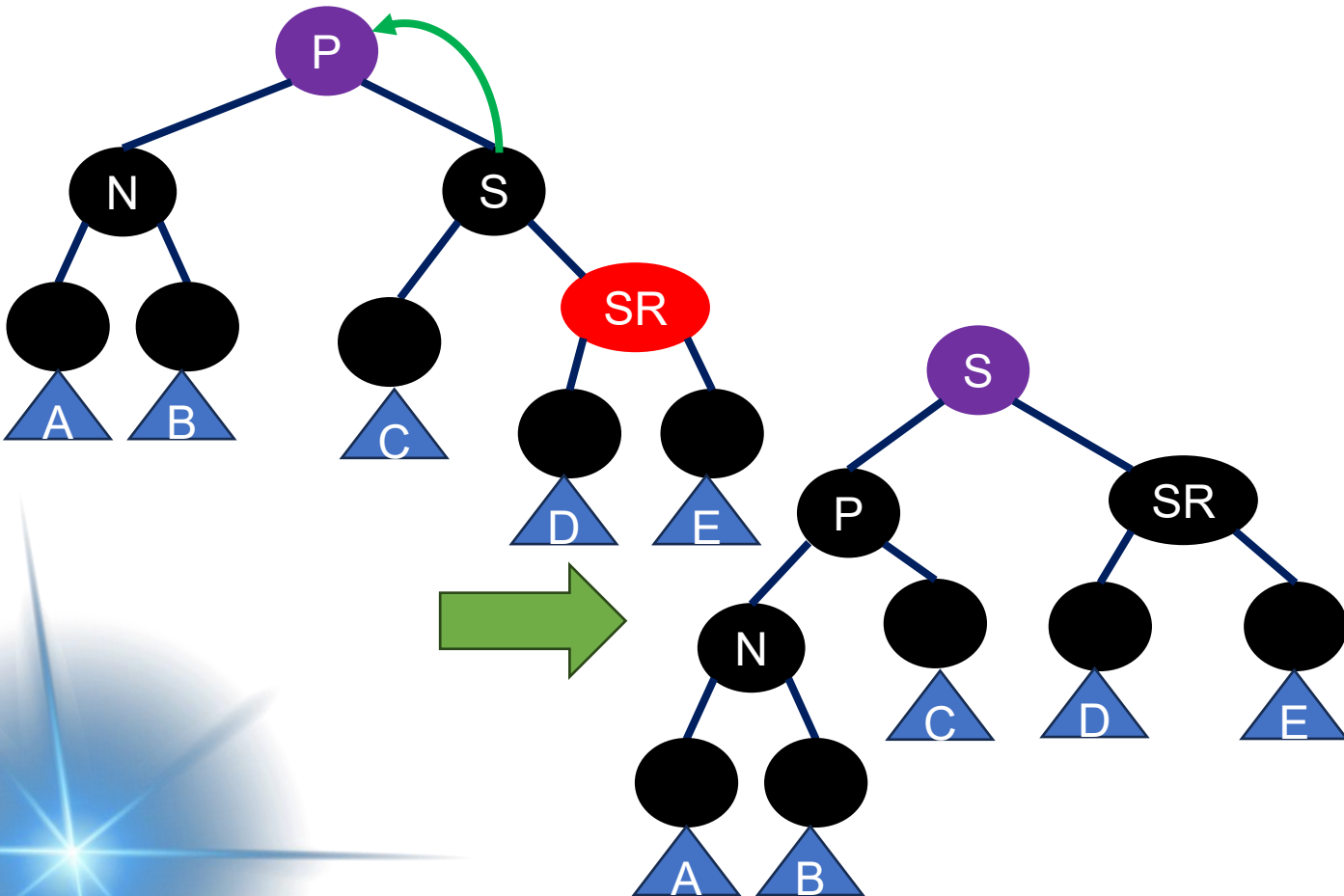


```
void del_case5(struct node *n)
{
    struct node *s;
    s = sibling(n);
    if (s->colour == BLACK)
    {
        if ((n == n->parent->left) &&
            (s->right->colour == BLACK) &&
            (s->left->colour == RED))
            rotate_right(s);
        else if ((n == n->parent->right) &&
                 (s->left->colour == BLACK) &&
                 (s->right->colour == RED))
            rotate_left(s);
        s->colour = RED;
        s->right->colour = BLACK;
    }
    del_case6(n);
}
```

Удаление узла из красно-черного дерева

Случай 4: S — черный, правый потомок S — красный, а N — левый потомок его родителя P

В этом случае выполняется поворот влево в точке P , чтобы сделать S родительским узлом P и правым потомком S . После поворота цвета P и S меняются местами, а правый потомок S окрашивается в чёрный цвет.



```
void del_case6(struct node *n)
{
    struct node *s;
    s = sibling(n);
    s->colour = n->parent->colour;
    n->parent->colour = BLACK;
    if (n == n->parent->left)
    {
        s->right->colour = BLACK;
        rotate_left(n->parent);
    }
    else
    {
        s->left->colour = BLACK;
        rotate_right(n->parent);
    }
}
```

Применение красно-черных деревьев

Красно-черные деревья являются эффективными бинарными деревьями поиска, поскольку они предлагают наихудшую гарантию времени для операций вставки, удаления и поиска. Красно-черные деревья не только ценны в чувствительных ко времени приложениях, таких как приложения реального времени, но также предпочтительны для использования в качестве строительного блока в других структурах данных, которые предоставляют наихудшую гарантию. Деревья AVL также поддерживают операции поиска, вставки и удаления $O(\log n)$, но они более жестко сбалансированы, чем красно-черные деревья, что приводит к более медленной вставке и удалению, но более быстрому извлечению данных.



Декартовы деревья (Cartesian trees)

(k, p) – вставляем ключ и приоритет

k – key; p – priority;

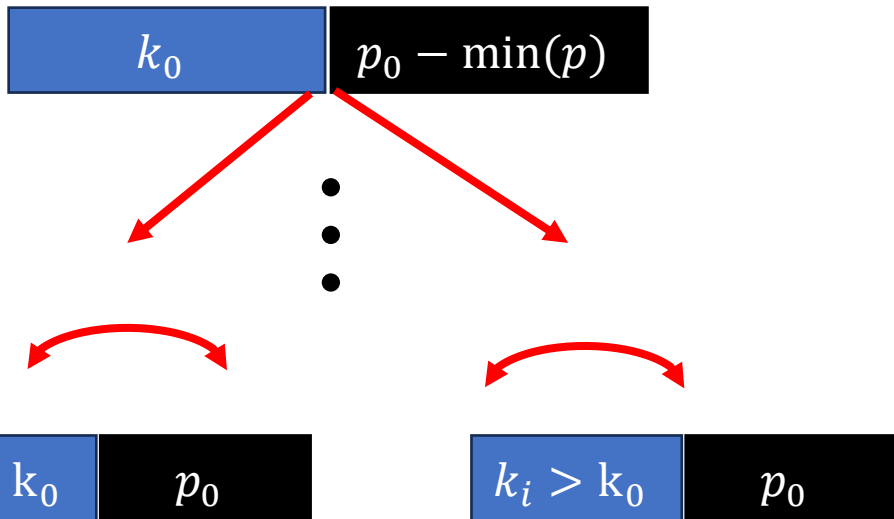
Свойство:

- 1) BST относительно ключей
- 2) Heap относительно приоритетов

Теорема 14.1. Существует единственное декартово дерево.

Доказательство: индукция.

1 элемент корень – очевидно, что он однозначно определен. (k_0, p_0) , $p_0 = \min$



Какова скорость работы?

Так как дерево единственно, кажется опасно.

Можно взять приоритет, как случайное число $[0, 1]$

Treap = tree + heap

Декартовы деревья (Cartesian trees)

(k, p) – вставляем ключ и приоритет

k – key; p – priority;

Мы имеем поиск (lookup), вставка (insert), удаление (remove)

Однако мы имеем классные операции слияние (merge) и разделение (split) за $O(\log(n))$

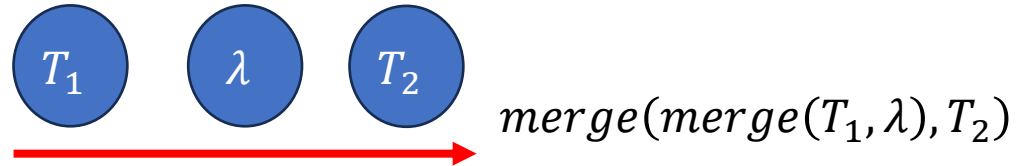
$merge(T_1, T_2) = T \quad \forall x \in T_1 \quad \forall y \in T_2 \quad k(x) < k(y)$

$split(T, \lambda) = (T_1 < \lambda, \lambda | \emptyset, T_2 > \lambda)$ - 3 дерева

Как будет работать insert, remove?

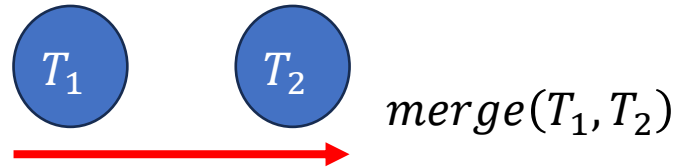
insert:

$T \xrightarrow{split} (T_1 < \lambda, \lambda | \emptyset, T_2 > \lambda) \xrightarrow{insert} (T_1 < \lambda, \lambda, T_2 > \lambda)$



remove:

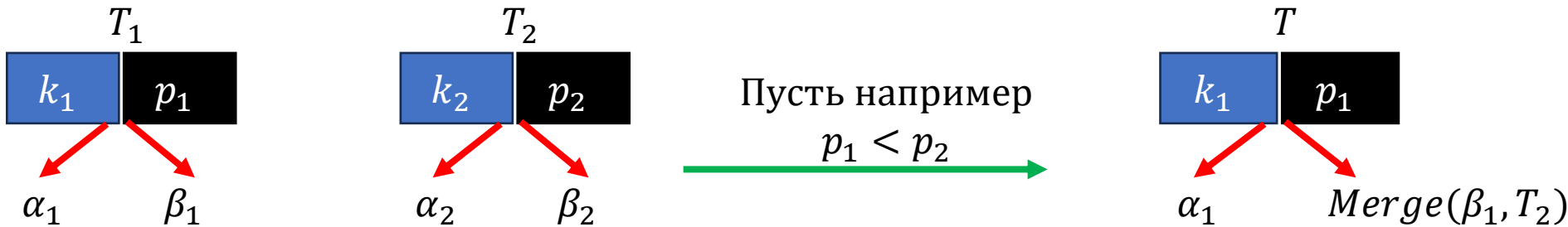
$T \xrightarrow{split} (T_1 < \lambda, \lambda | \emptyset, T_2 > \lambda) \xrightarrow{insert} (T_1 < \lambda, \emptyset, T_2 > \lambda)$



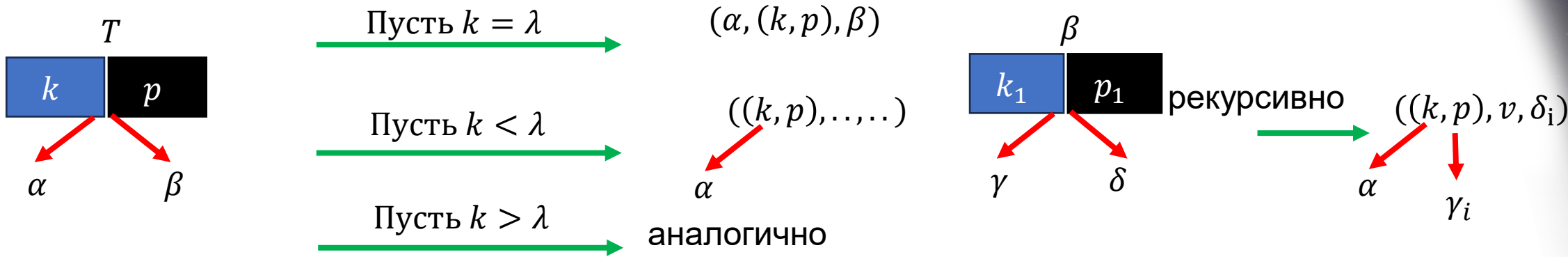
Декартовы деревья (Cartesian trees)

(k, p) – вставляем ключ и приоритет
 k – key; p – priority;

$$merge(T_1, T_2) = T \quad \forall x \in T_1 \quad \forall y \in T_2 \quad k(x) < k(y)$$



$$split(T, \lambda) = (T_1 < \lambda, \lambda | \emptyset, T_2 > \lambda)$$



Амортизированные балансировки

Поиск в худшем случае $\log(n)$

Вставка, удаление в среднем случае $\log(n)$

Примеры:

- Splay Tree
- Scapegoat Tree



Scapegoat Tree

Размер поддеревя $s(v) = \# \text{subtree}(v)$

$$\forall v \quad s(l) \leq \alpha s(v) \quad s(r) \leq \alpha s(v) \quad \alpha \in (\frac{1}{2}, 1)$$

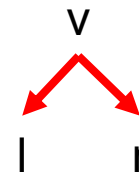
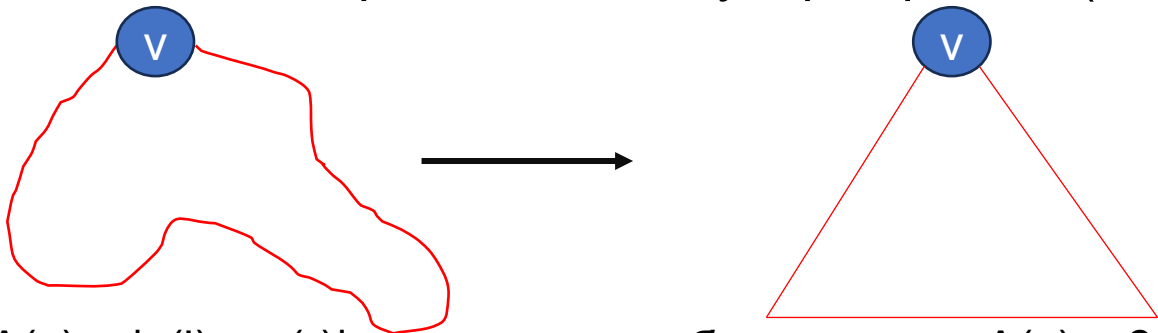
Храним размеры поддеревьев

Insert

Инкрементируем при проходе до x все поддеревья

Проходим весь путь до вершины x и ищем «козла отпущения»

Из плохого дерева делаем супер хорошее (сбалансированное).



Покажем, что «козел отпущения встречается настолько резка, что стоимость $\log$.

$\Delta(v) = |s(l) - s(r)|$, возникает проблема, когда $\Delta(v) \geq \beta s(v) > 1/2 s(v)$

При балансировке – проблема с равным количеством $\rightarrow$ минимизируем дисбаланс

Делаем, чтобы дисбаланс в каждой вершине был не более 1, тогда половина вершин

сбалансирована $\sum_x \Delta(x) \leq \frac{s(v)}{2}$. Для грубой оценки, для выполнения условий $\alpha = 0.9$, $\beta = 0.8 > 0.5$.

Можно не хранить размер поддеревьев – «козел отпущения» тоже долго считается.

Если нет «козлов отпущения» - $H(\text{subtree}(v)) = O(\log(s(v)))$ – новые условия вместо α и β .

Расширенные (Splay) деревья

Деревья Splay были изобретены Дэниелом Слейтором и Робертом Тарьяном. Splay дерево — это самобалансирующееся бинарное дерево поиска с дополнительным свойством, которое заключается в том, что к недавно использованным элементам можно быстро получить повторный доступ. Оно считается эффективным бинарным деревом, поскольку выполняет основные операции, такие как вставка, поиск и удаление, за амортизированное время $O(\log(n))$. Для многих неравномерных последовательностей операций расширяющиеся деревья работают лучше, чем другие деревья поиска, даже если конкретный шаблон последовательности неизвестен.

Splay дерево состоит из бинарного дерева без дополнительных полей. Когда осуществляется доступ к узлу в расширяющемся дереве, он поворачивается или «расширяется» к корню, тем самым изменяя структуру дерева. Поскольку наиболее часто используемый узел всегда перемещается ближе к начальной точке поиска (или корневому узлу), эти узлы, следовательно, находятся быстрее. Простая идея заключается в том, что если осуществляется доступ к элементу, то, скорее всего, к нему будет осуществлен доступ снова. В Splay дереве такие операции, как вставка, поиск и удаление, объединены с одной базовой операцией, называемой развертыванием. Развертывание дерева для определенного узла перестраивает дерево так, чтобы поместить этот узел в корень. Методика для этого заключается в том, чтобы сначала выполнить стандартный поиск двоичного дерева для этого узла, а затем использовать вращения в определенном порядке, чтобы переместить узел наверх.

Операции над развернутыми деревьями

четыре основные операции, которые выполняются над
splay деревом. К ним относятся

- развертывание
- вставка
- поиск
- удаление.



Развертывание

Когда мы получаем доступ к узлу N , развертывание выполняется над N , чтобы переместить его в корень. Для выполнения операции развертывания выполняются определенные шаги развертывания, где каждый шаг перемещает N ближе к корню. Развертывание определенного интересующего узла после каждого доступа гарантирует, что недавно использованные узлы остаются ближе к корню, а дерево остается примерно сбалансированным, так что могут быть достигнуты желаемые амортизированные временные границы. Каждый шаг расширения зависит от трех факторов:

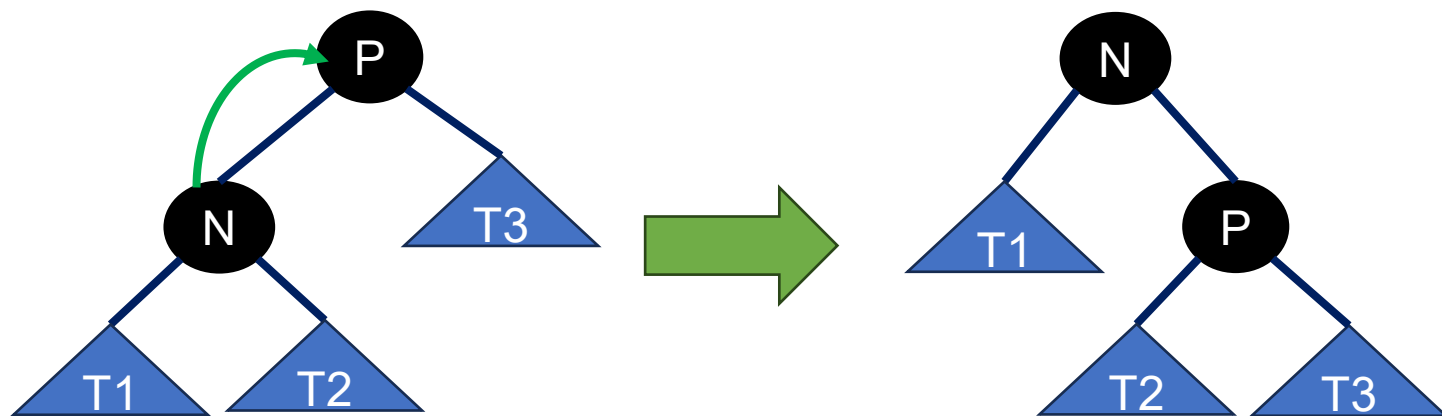
- Является ли N левым или правым потомком своего родителя P ,
- Является ли P корнем или нет, и если нет,
- Является ли P левым или правым потомком своего родителя, G (дед N). В зависимости от этих трех факторов у нас есть один шаг расширения на основе каждого фактора.



Развертывание

Шаг зиг

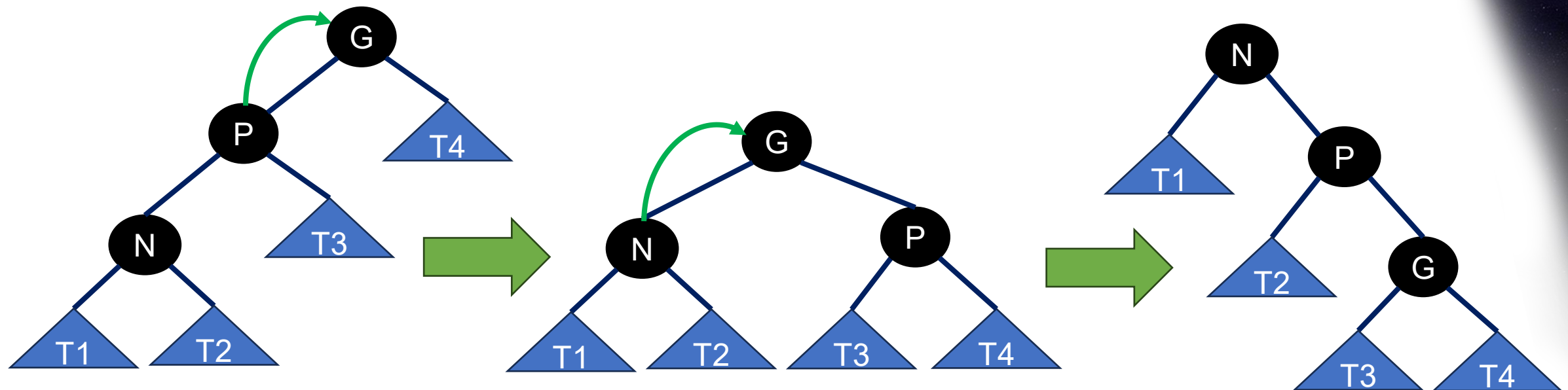
Операция зигзага выполняется, когда Р (родитель N) является корнем дерева расширения. На шаге зигзага дерево поворачивается на ребре между N и Р. Шаг зигзага обычно выполняется как последний шаг в операции расширения и только когда N имеет нечетную глубину в начале операции.



Развертывание

Шаг зигзиг

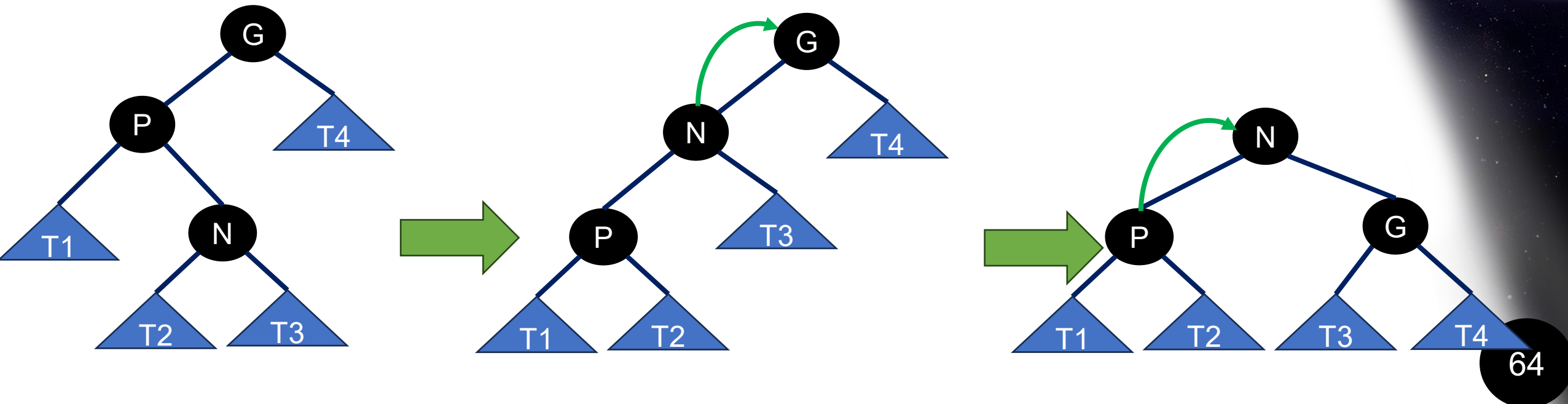
Операция зигзага выполняется, когда P не является корнем. В дополнение к этому, N и P являются либо правыми, либо левыми потомками своих родителей. На рисунке показан случай, когда N и P являются левыми потомками. Во время шага зигзаг сначала дерево поворачивается на ребре, соединяющем P и его родителя G , а затем снова поворачивается на ребре, соединяющем N и P .



Развертывание

Шаг зигзаг

Операция зигзаг выполняется, когда P не является корнем. В дополнение к этому, N является правым потомком P , а P является левым потомком G или наоборот. При шаге зигзаг дерево сначала поворачивается на ребре между N и P , а затем поворачивается на ребре между N и G .

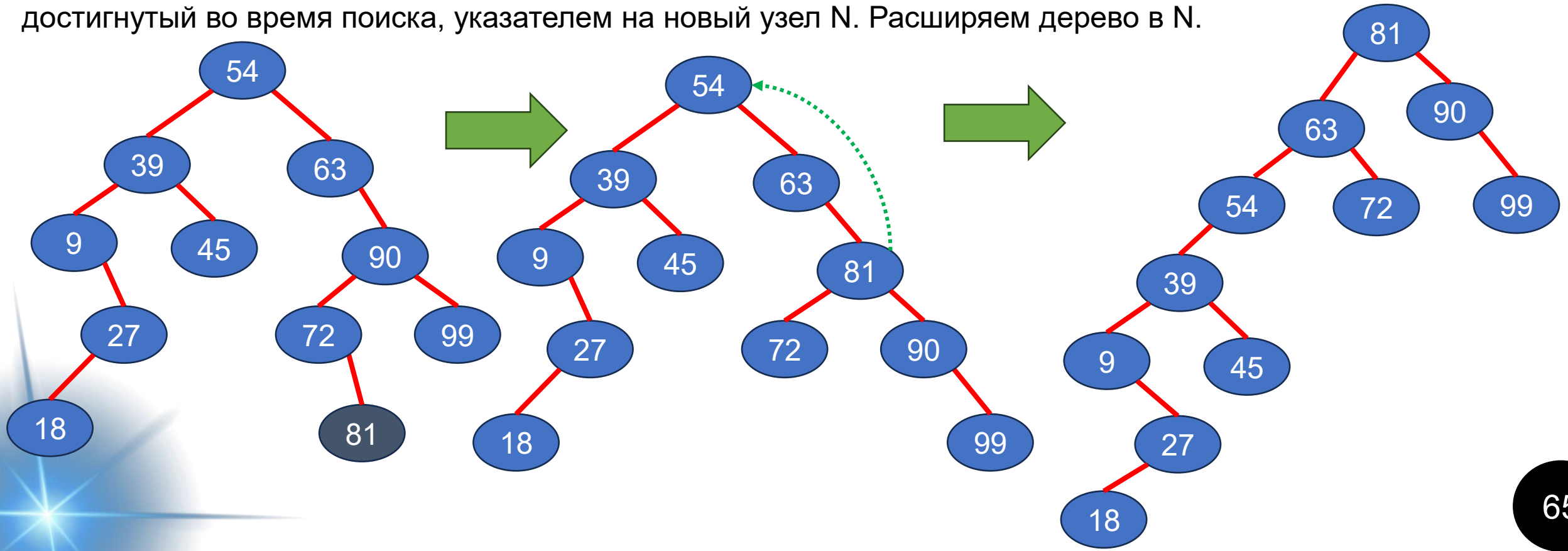


Вставка узла в splay дерево

Хотя процесс вставки нового узла N в расширяющееся дерево начинается так же, как мы вставляем узел в бинарное дерево поиска, но после вставки N становится новым корнем расширяющегося дерева. Шаги, выполняемые для вставки нового узла N в расширяющееся дерево, можно представить следующим образом:

Шаг 1 Поиск N в расширяющемся дереве. Если поиск успешен, расширяем в узле N.

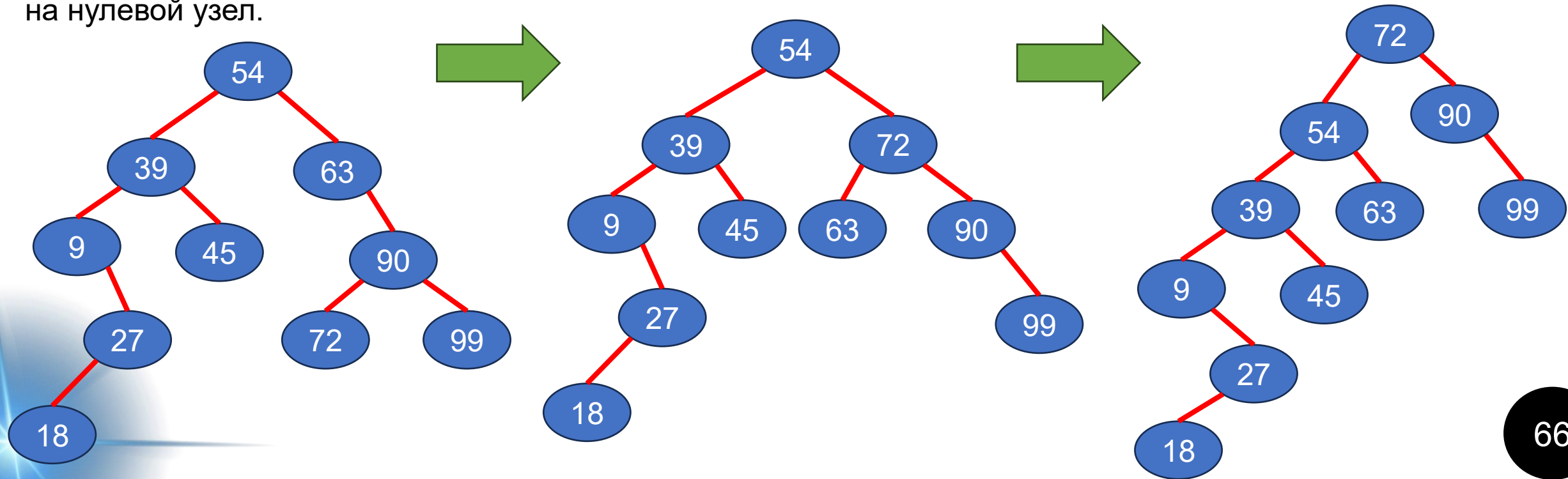
Шаг 2 Если поиск не удался, добавляем новый узел N таким образом, чтобы он заменил указатель NULL, достигнутый во время поиска, указателем на новый узел N. Расширяем дерево в N.



Поиск узла в splay дереве

Если в раскрывающемся дереве присутствует определенный узел N, то возвращается указатель на N; в противном случае возвращается указатель на нулевой узел. Шаги, выполняемые для поиска узла N в раскрывающемся дереве, включают:

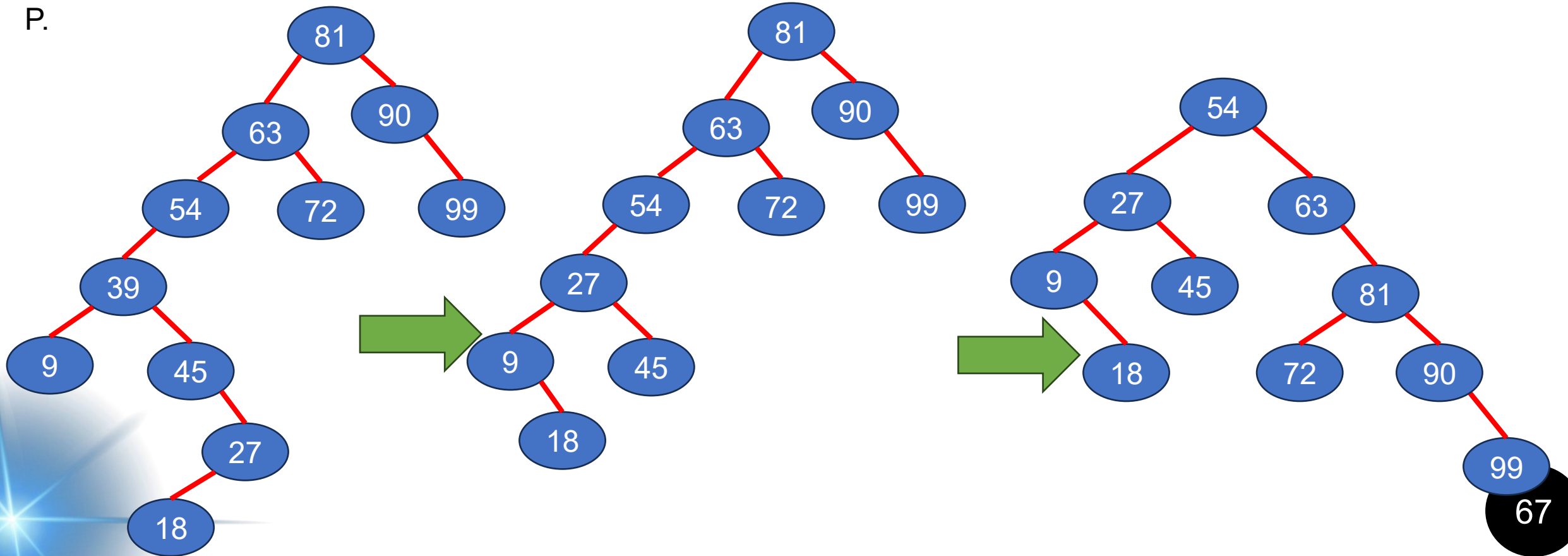
- Просмотрите корень разветвленного дерева, ища N.
- Если поиск успешен и мы достигли N, то растягиваем дерево в N и возвращаем указатель на N.
- Если поиск не удался, т. е. растягиваемое дерево не содержит N, то мы достигли нулевого узла. Растягиваем дерево в последнем ненулевом узле, достигнутом во время поиска, и возвращаем указатель на нулевой узел.



Поиск узла в splay дереве

Чтобы удалить узел N из растягиваемого дерева, мы выполняем следующие шаги:

- Поиск N, который необходимо удалить. Если поиск не удался, растягиваем дерево в последнем ненулевом узле, обнаруженном во время поиска.
- Если поиск успешен и N не является корневым узлом, то пусть P будет родителем N. Замените N соответствующим потомком P (как мы делаем в бинарном дереве поиска). Наконец, растягиваем дерево в P.



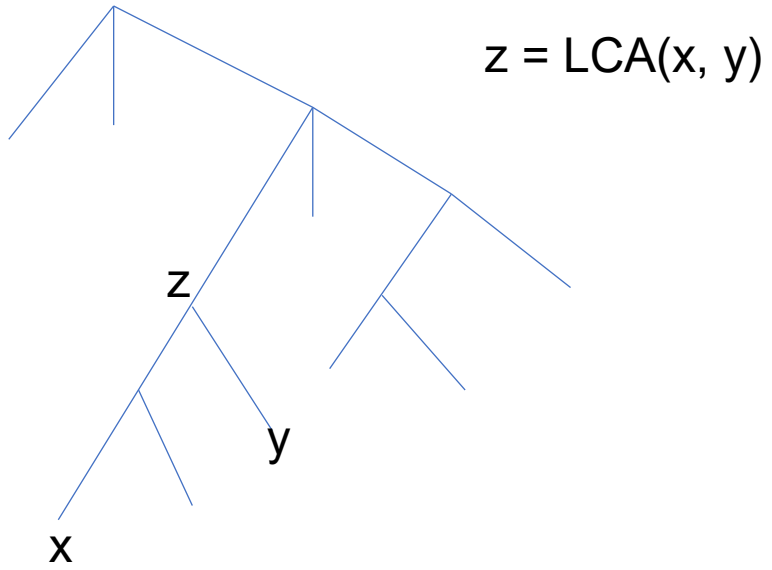
Преимущества использования расширяющегося дерева

- Расширяющееся дерево обеспечивает хорошую производительность для операций поиска, вставки и удаления. Это преимущество основано на том факте, что расширяющееся дерево является самобалансирующейся и самооптимизирующейся структурой данных, в которой часто используемые узлы перемещаются ближе к корню, чтобы к ним можно было быстро получить доступ. Это преимущество особенно полезно для реализации кэшей и алгоритмов сборки мусора.
- Расширяющиеся деревья значительно проще в реализации, чем другие самобалансирующиеся бинарные деревья поиска, такие как красно-черные деревья или деревья AVL, в то время как их средняя производительность так же эффективна.
- Расширяющиеся деревья минимизируют требования к памяти, поскольку они не хранят никаких бухгалтерских данных.
- В отличие от других типов самобалансирующихся деревьев, расширяющиеся деревья обеспечивают хорошую производительность (амортизированная $O(\log n)$) с узлами, содержащими идентичные ключи.

Недостатки использования расширяющегося дерева

- При последовательном доступе ко всем узлам дерева в отсортированном порядке результирующее дерево становится полностью несбалансированным. Это требует n доступов к дереву, в которых каждый доступ занимает $O(\log n)$ времени. Например, повторный доступ к первому узлу запускает операцию, которая, в свою очередь, занимает $O(n)$ операций для повторной балансировки дерева перед возвратом первого узла. Хотя это создает значительную задержку для конечной операции, амортизированная производительность по всей последовательности по-прежнему составляет $O(\log n)$.
- Для равномерного доступа производительность расширяющегося дерева будет значительно хуже, чем несколько сбалансированного простого двоичного дерева поиска. Для равномерного доступа, в отличие от расширяющихся деревьев, эти другие структуры данных обеспечивают гарантии времени в худшем случае и могут быть более эффективными в использовании.

LCA (least common ancestor)



RMQ (range minimum query)

$$A = (a_0, a_1, \dots, a_{n-1}), RMQ = \min_{i < k < j} a_k$$

Хотим искать быстро, нужен какой-то быстрый препроцессинг

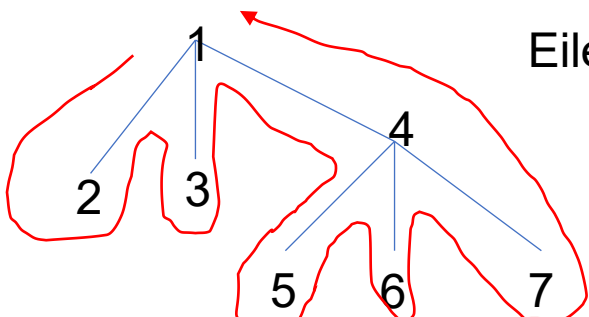
Будет пара (t_{prepr}, t_{query})

Редукция

LCA → RMQ

t_{prepr}	t_{query}	name
$O(n^2)$	$O(1)$	trivial
$O(n)$	$O(1)$	

Euler Tour



id	1	2	1	3	1	4	5	4	6	4	7	4	1
deeps	0	1	0	1	0	1	2	1	2	1	2	1	0

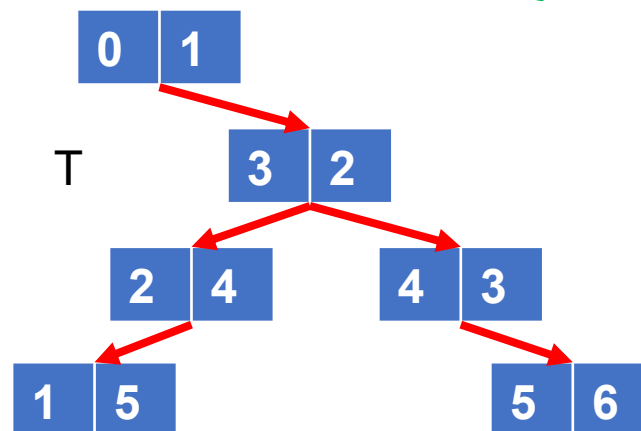
RMQ → LCA

Используем декартово дерево

Имеем массив A

(берем приоритеты):

0	1	2	3	4	5	key
1	5	4	2	3	6	priority



$z = LCA(x, y)$

Доказательство эквивалентности д/з. Спрашивать при сдаче 11 задачи 12 пака. + построение декартова дерева за $O(n)$

$LCA(3, 6) = 1$
 $LCA(2, 4) = 0$

Получился отрезок, ищем число с минимальной глубиной

Решение LCA и RMQ

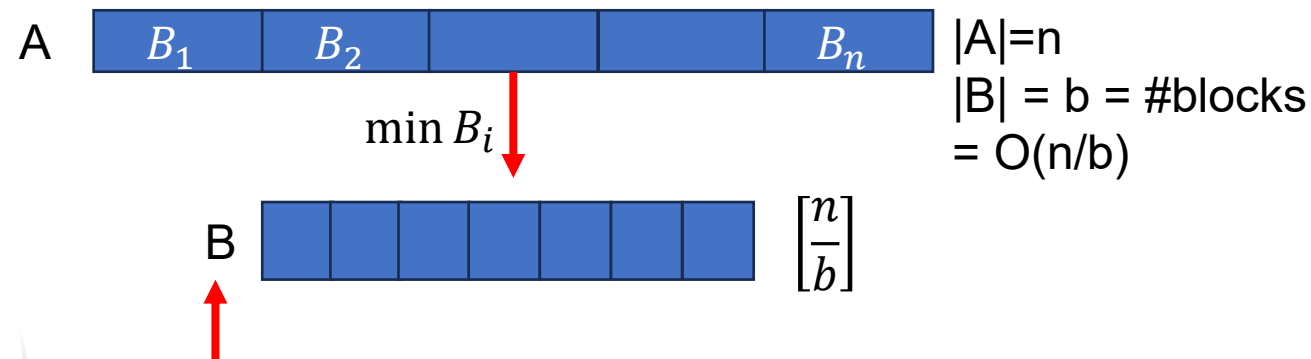
Дана любая из задач. Делаем преобразование
 $RMQ \rightarrow LCA \rightarrow RMQ(\pm 1)$, то есть $|a_i - a_{i+1}| = 1$

Шаг 1: Sparse precompute

$$M(i, k) = \operatorname{argmin}_{i \leq j < i+2^k} a_j$$

$$M(i, k+1) = \operatorname{argmin}(M(i, k), M(i+2^k, k))$$

Шаг 2: Сжать массив в log раз



Воспользуемся знанием, что у нас $RMQ(\pm 1)$ и то что при вычитании const минимум останется в том же месте

Вычтем $B_i[0]$



$$\# \text{various blocks} = 2^{b-1}$$

$$b = 1/2 \log n$$

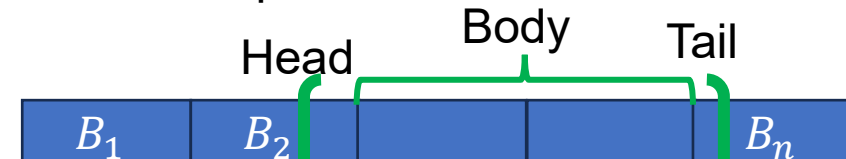
Сколько памяти нужно (+ количество входов · сколько подзадач) = $\frac{n}{b} \log \frac{n}{b} + 2^{b-1} b^2 = O(n + \sqrt{n} \log^2 n)$

Можно в виде целых чисел разные варианты представлять и удобно потом получать ответ, разлагая по степени 2.

t_{prepr}	t_{query}	name
$O(n^2)$	$O(1)$	trivial
$O(n \log(n))$	$O(1)$	sparse
$O(n)$	$O(1)$	Farach-Colton-Bender

Пусть $b \sim \log n$, тогда память $O\left(\frac{n}{b} \log \frac{n}{b}\right) = O(n)$

Проблема?



Запросы 3 штуки, Head, Tail - маленькие
 Проблема 2 – а что если нет Body?